Defense:
Prevent, Mitigate, Respond & Test

Taller de Seguridad

Ing. Martin Di Paola - Fiuba

BARINGS

Barings Bank
(english bank founded in 1762)

Nick Leeson

Trading Manager
and
his own Supervisor

2 roles in 1 person

no separation

no minimum

no monitoring

Nick Leeson

Trading Manager
and
his own Supervisor

2 roles in 1 person

no separation

no minimum

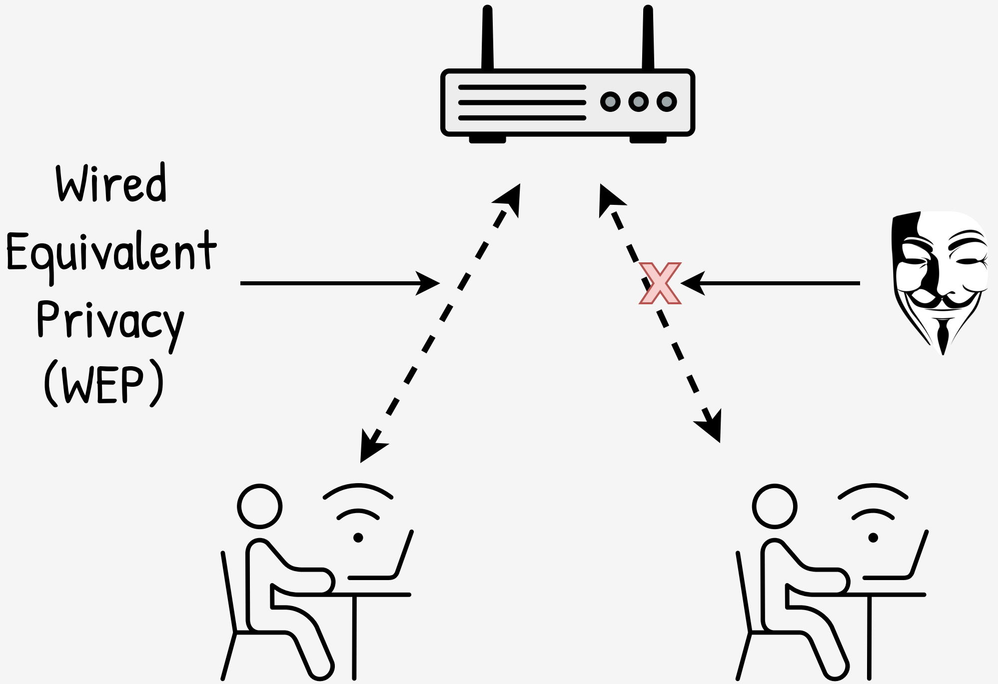
no monitoring

Losses for 1400 millon dolars

Bankruptcy in 1995; bank sold by 1 pound sterling

1762 – 1995 (264 years)

Nick Leeson, captured in Kuala Lumpur


Wired
Equivalent
Privacy
(WEP)

WEP

(closed-ish)

1997

(impl)

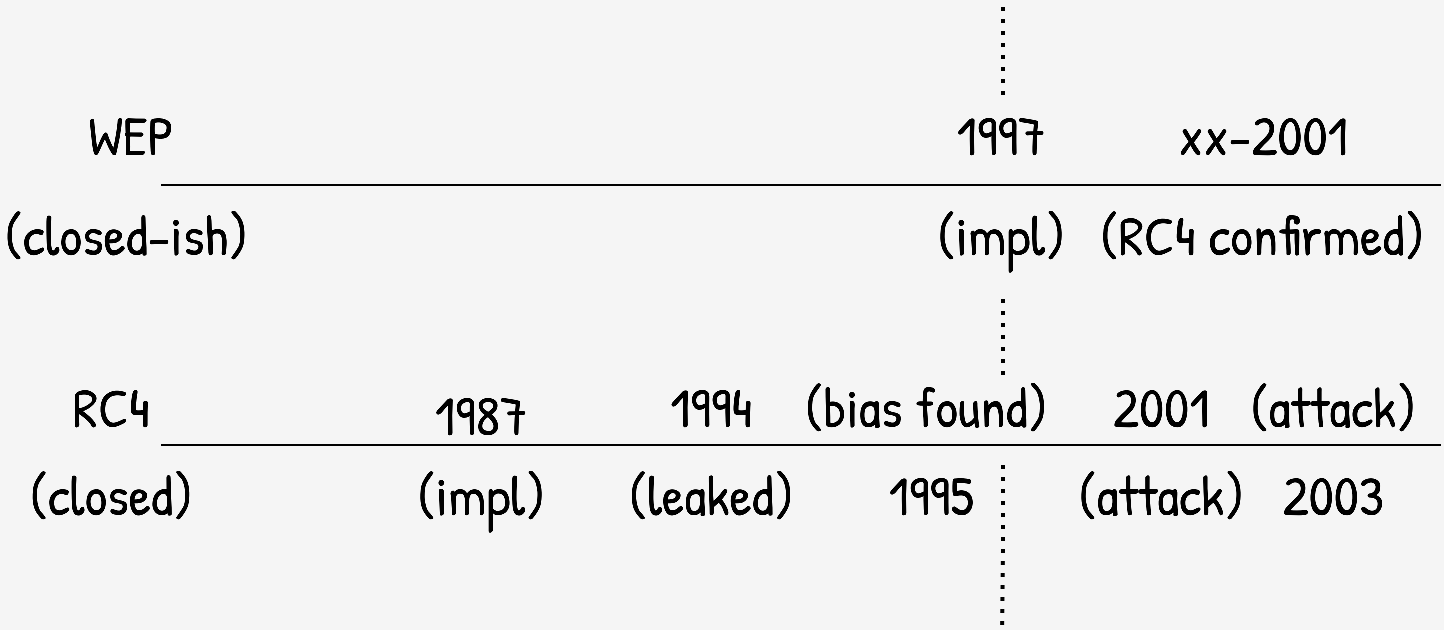
WEP	1997	xx-2001
(closed-ish)	(impl)	(RC4 confirmed)

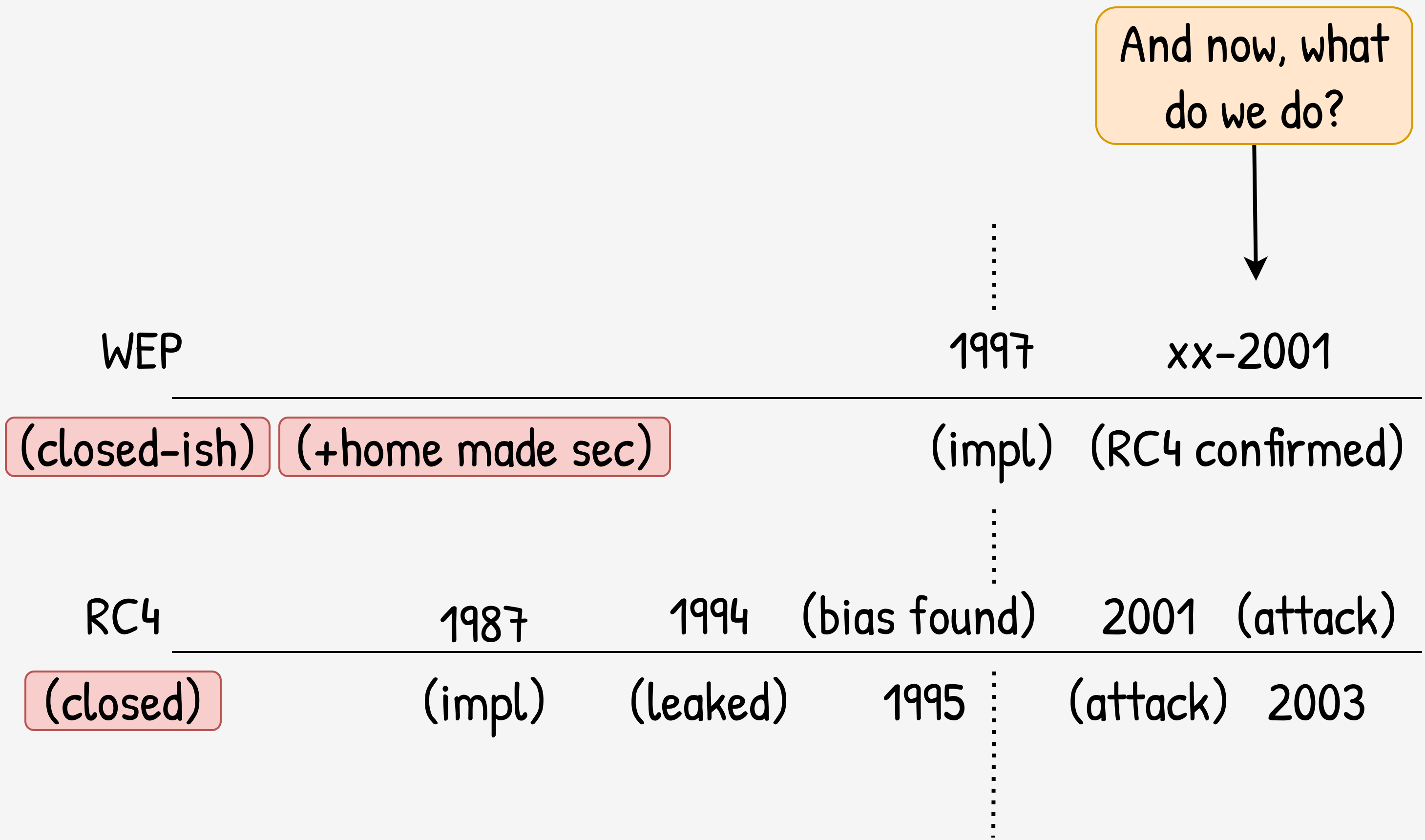
WEP 1997 xx-2001

(closed-ish) (impl) (RC4 confirmed)

RC4 1987 1994 (bias found)

(closed) (impl) (leaked) 1995


WEP
1997
xx-2001
(closed-ish)
(impl)
(RC4 confirmed)
RC4
1987
1994
(bias found)
2001
(attack)
(closed)
(impl)
(leaked)
1995
(attack)
2003

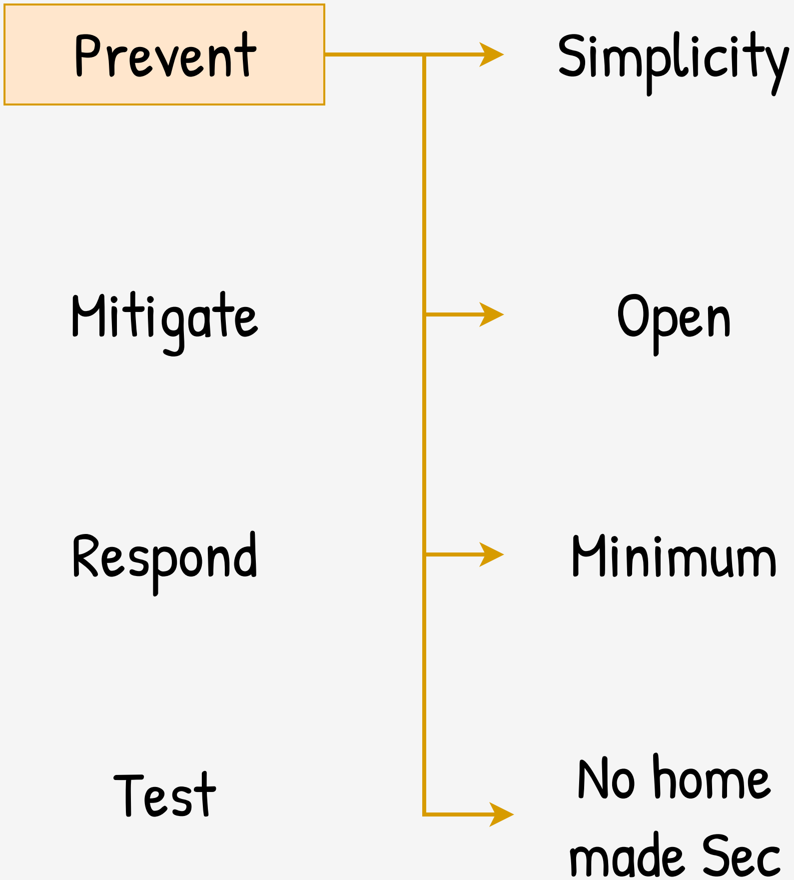

And now, what do we do?
WEP
1997
xx-2001
(closed-ish)
(+home made sec)
(impl)
(RC4 confirmed)
RC4
1987
1994
(bias found)
2001
(attack)
(closed)
(impl)
(leaked)
1995
(attack)
2003

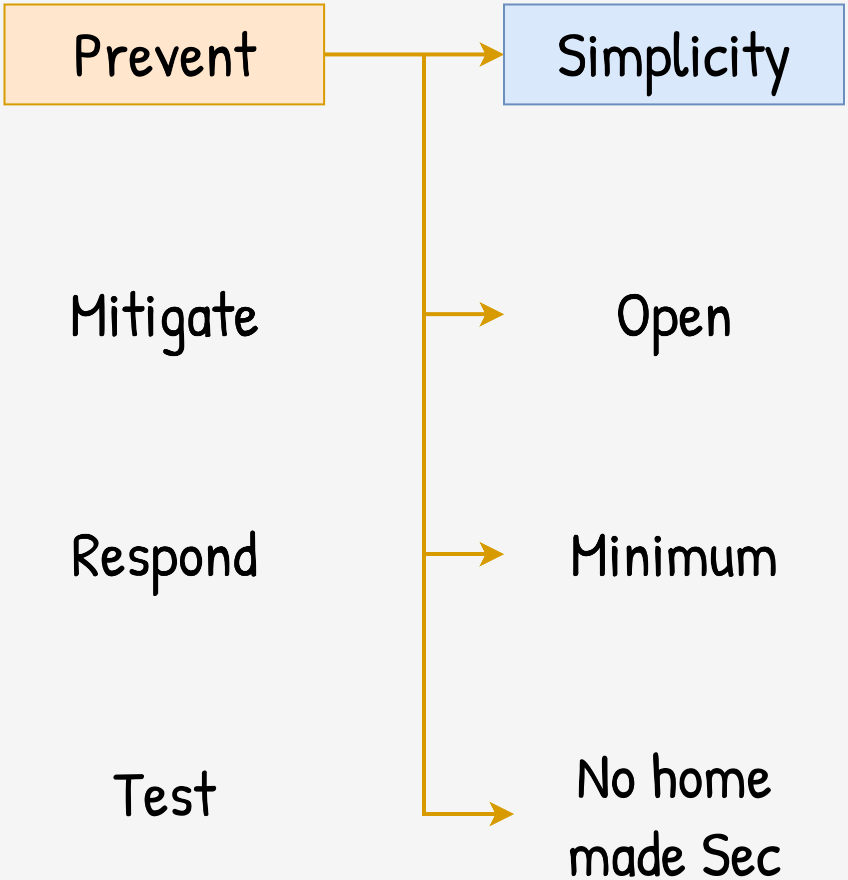
Prevent

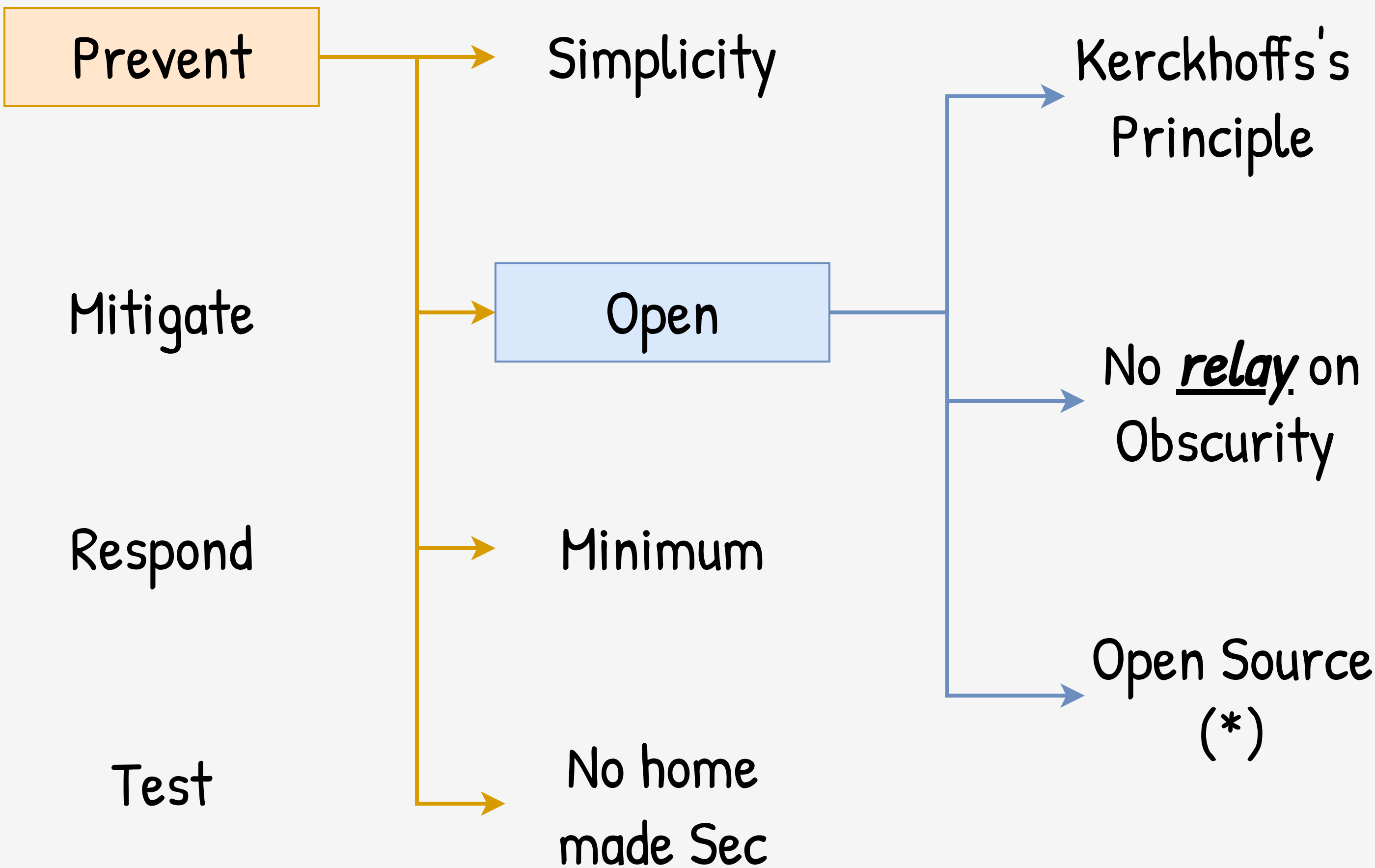
Mitigate

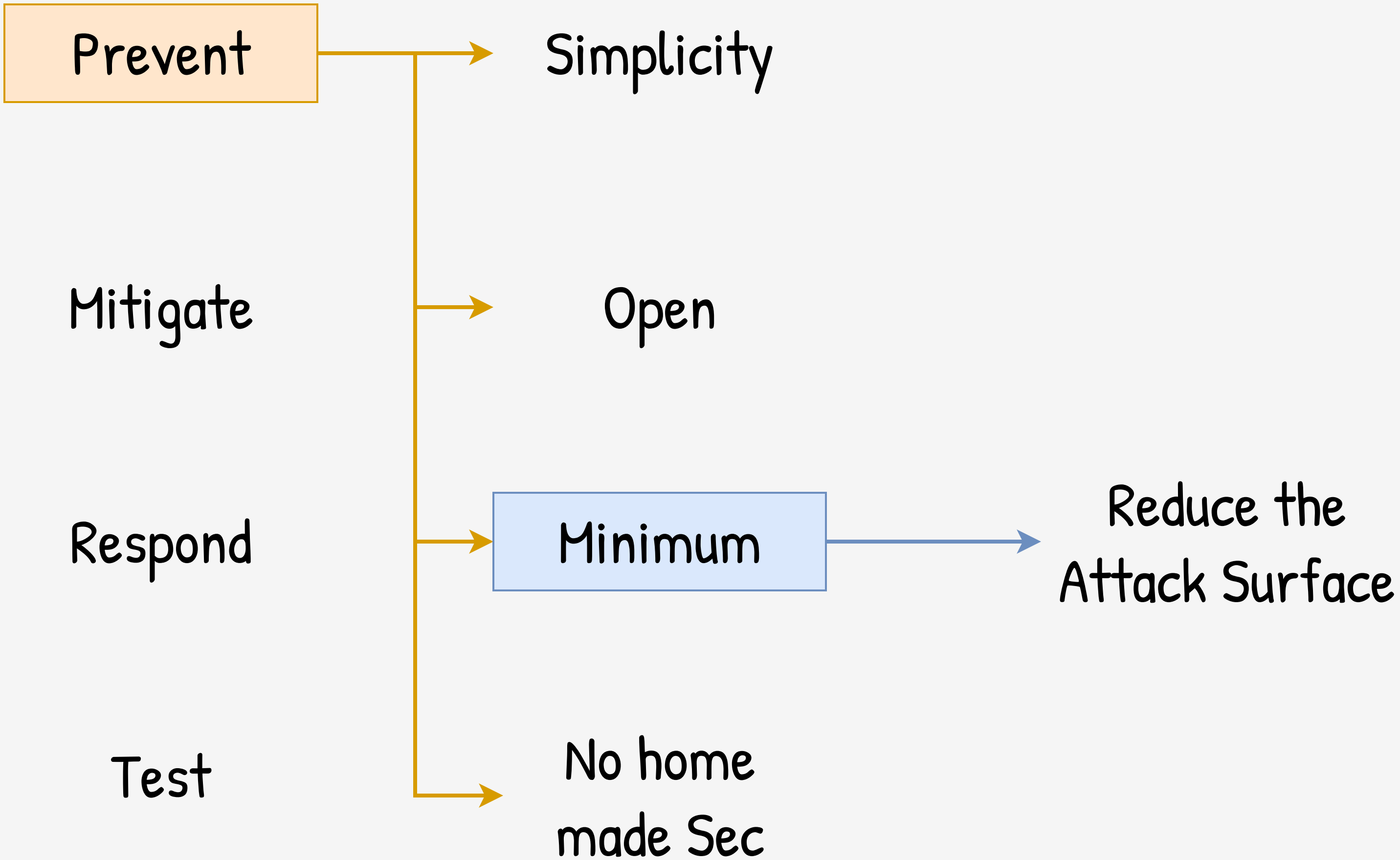
Respond

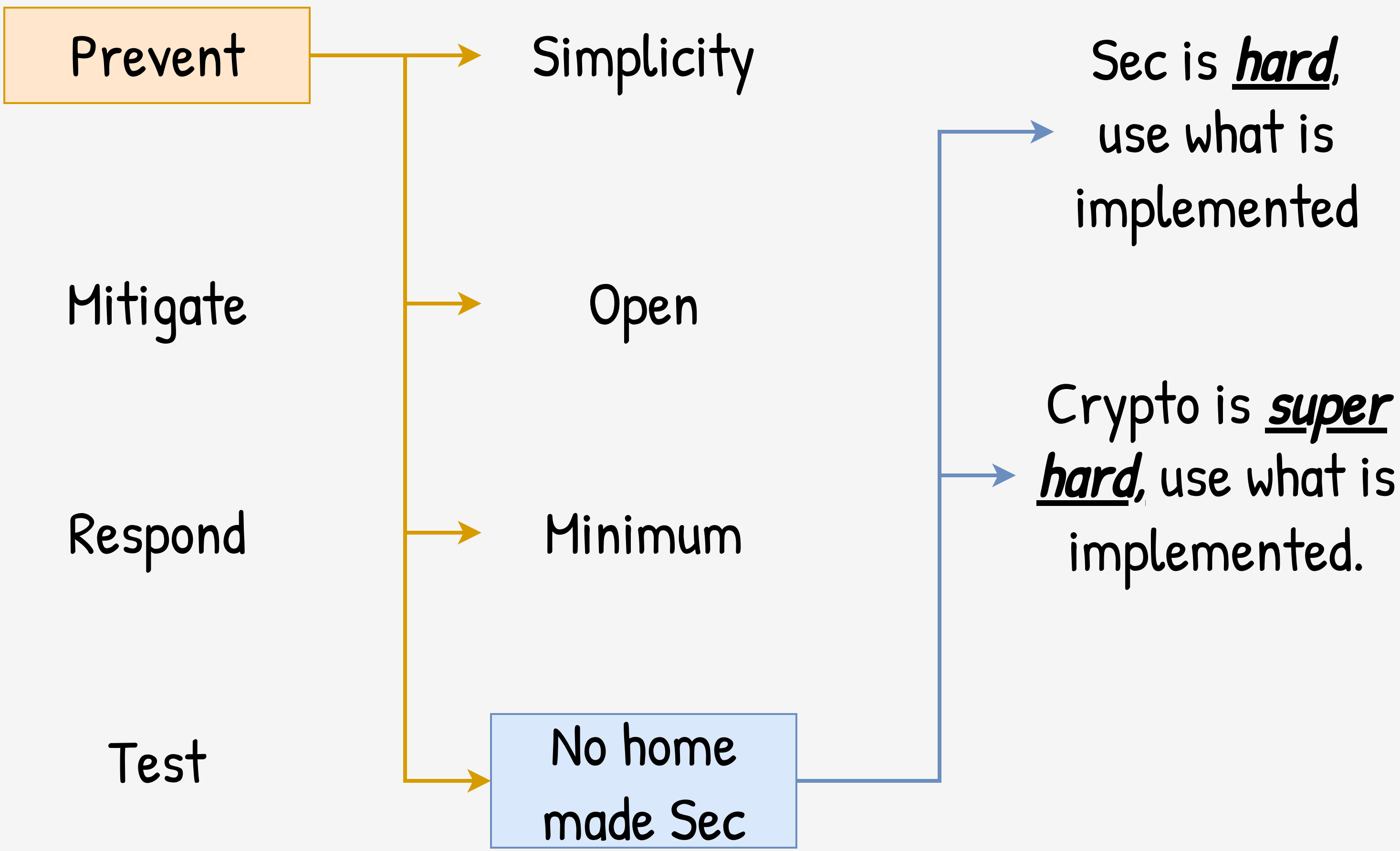
Test

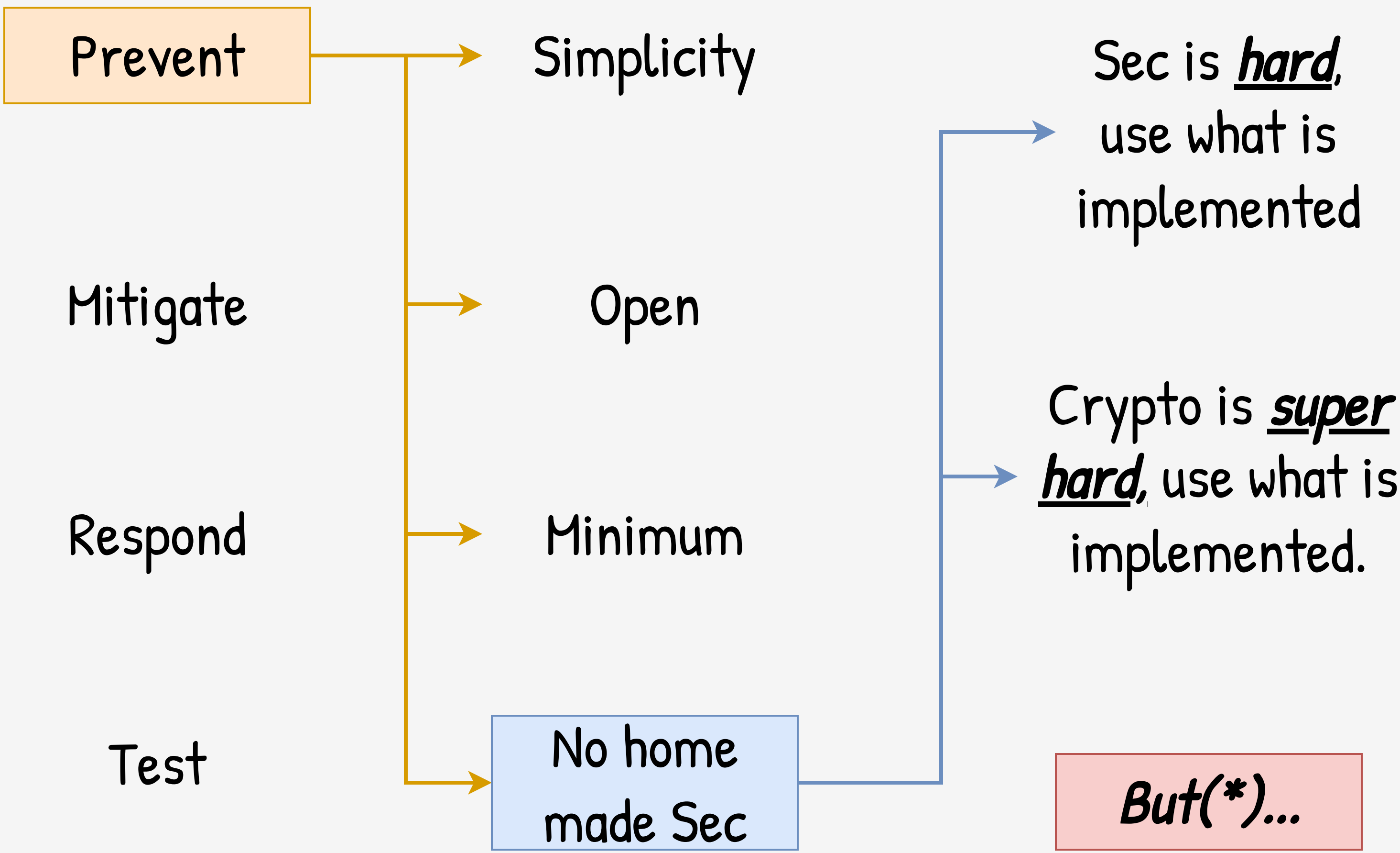

Prevent
Mitigate
Respond
Test
Simplicity
Open
Minimum
No home made Sec

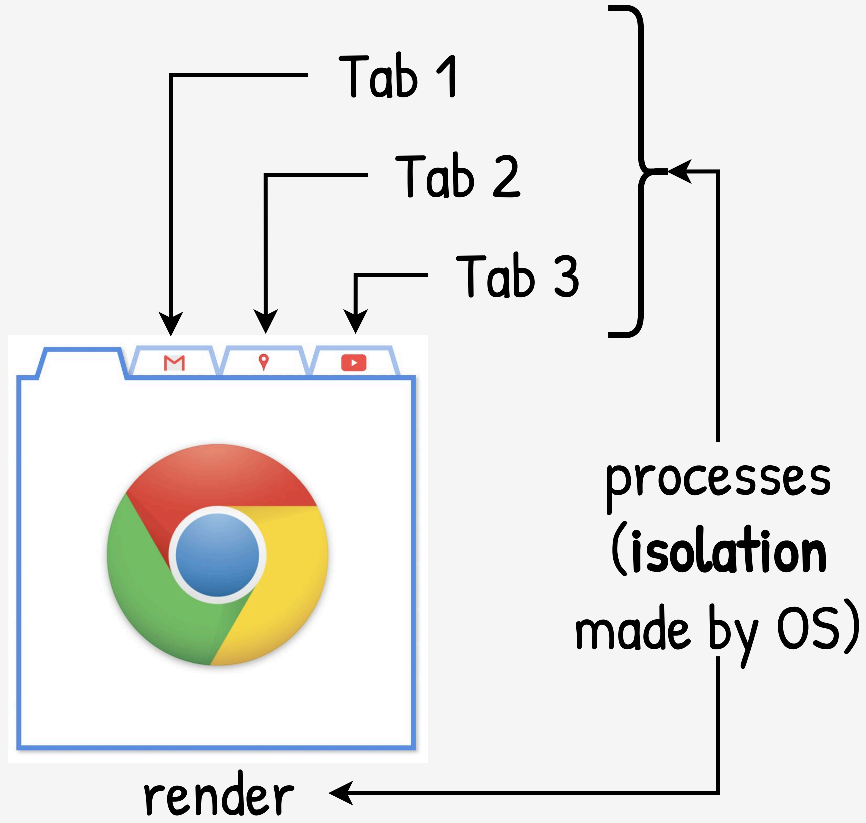

Prevent
Simplicity
Mitigate
Open
Respond
Minimum
Test
No home
made Sec

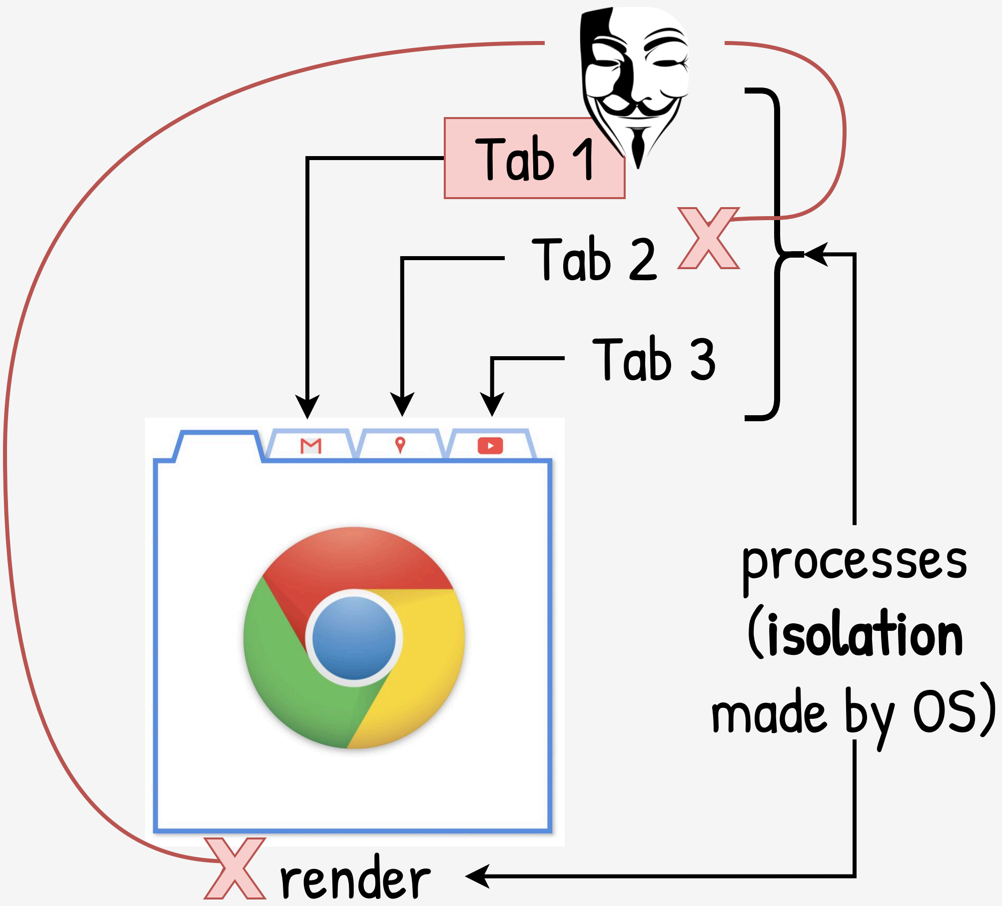

Prevent
Mitigate
Respond
Test
Simplicity
Open
Minimum
No home made Sec
Kerckhoffs's Principle
No relay on Obscurity
Open Source (*)


Prevent
Mitigate
Respond
Test
Simplicity
Open
Minimum
No home made Sec
Reduce the Attack Surface

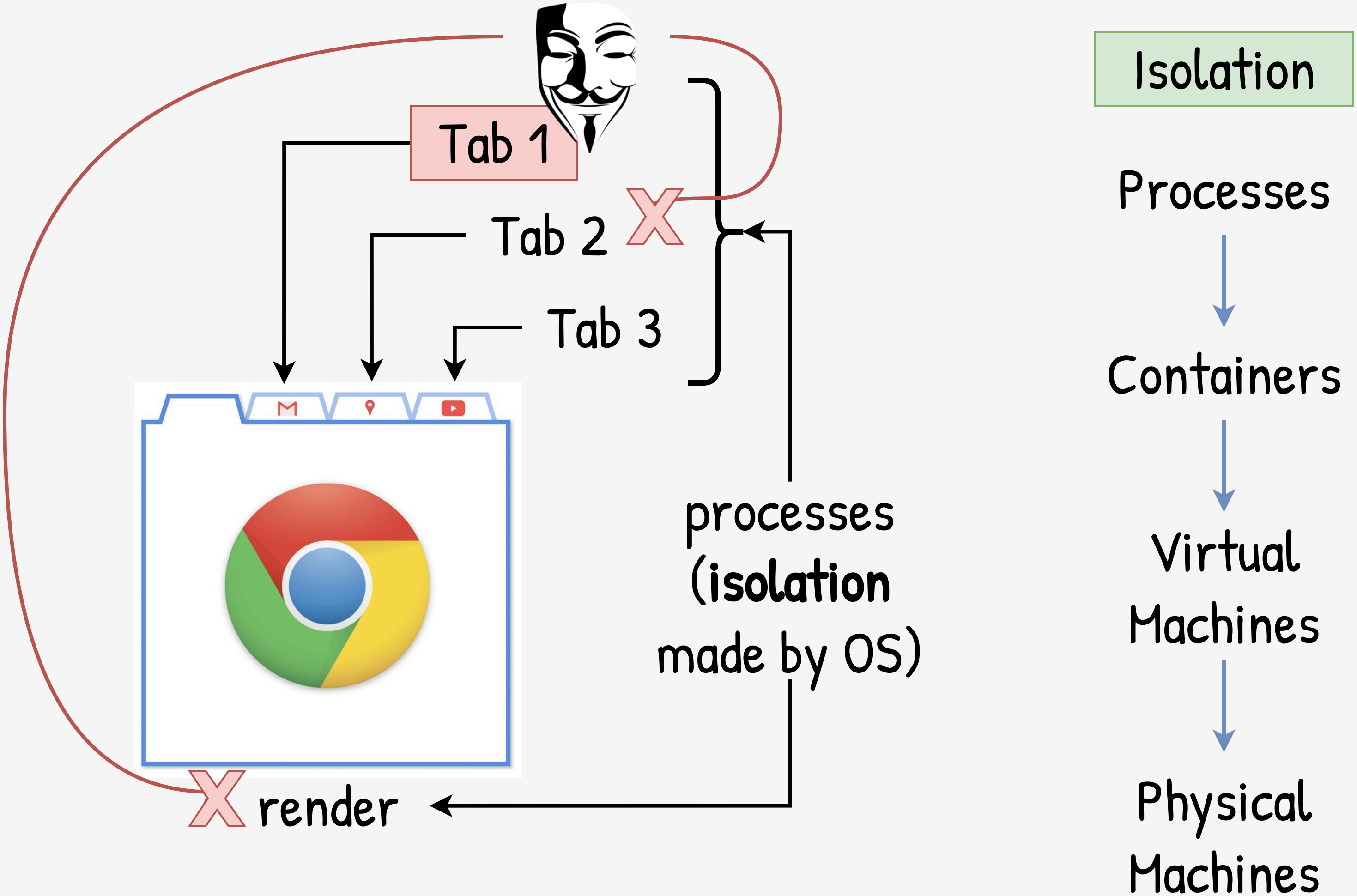

Prevent
Mitigate
Respond
Test
Simplicity
Open
Minimum
No home made Sec
Sec is hard, use what is implemented
Crypto is super hard, use what is implemented.

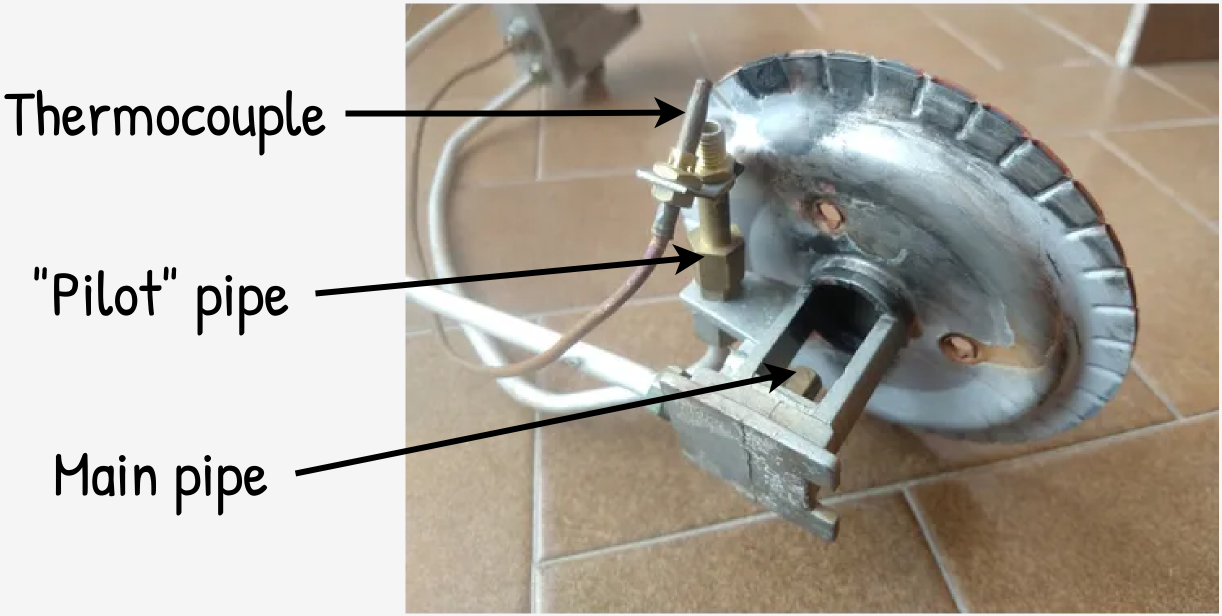

Prevent
Mitigate
Respond
Test
Simplicity
Open
Minimum
No home made Sec
Sec is ***hard***, use what is implemented
Crypto is ***super hard***, use what is implemented.
But(*)...


Tab 1
Tab 2
Tab 3
processes
(**isolation**
made by OS)
render



Tab 1
Tab 2
Tab 3
processes
(**isolation**
made by OS)
render

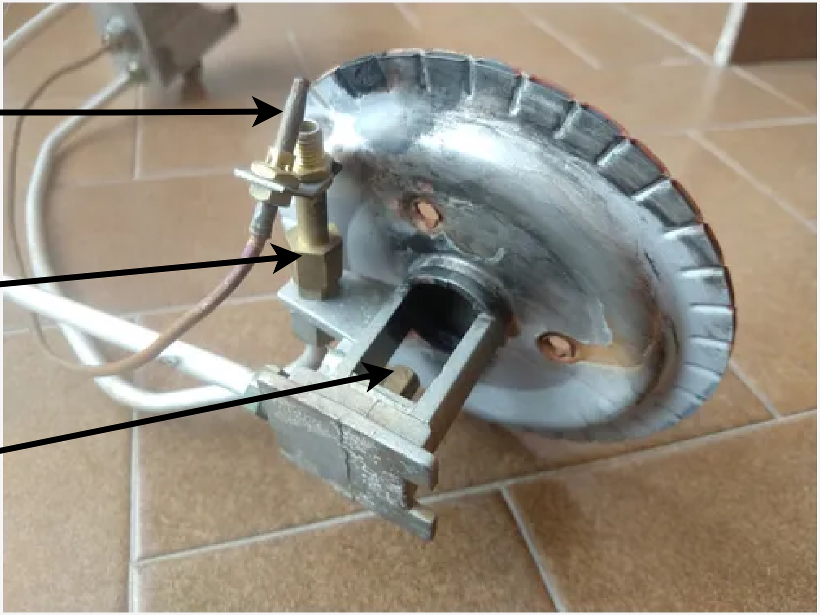

Tab 1
Tab 2
Tab 3
processes
(**isolation**
made by OS)
render
Isolation
Processes
Containers
Virtual
Machines
Physical
Machines


Thermocouple
"Pilot" pipe
Main pipe

Thermocouple

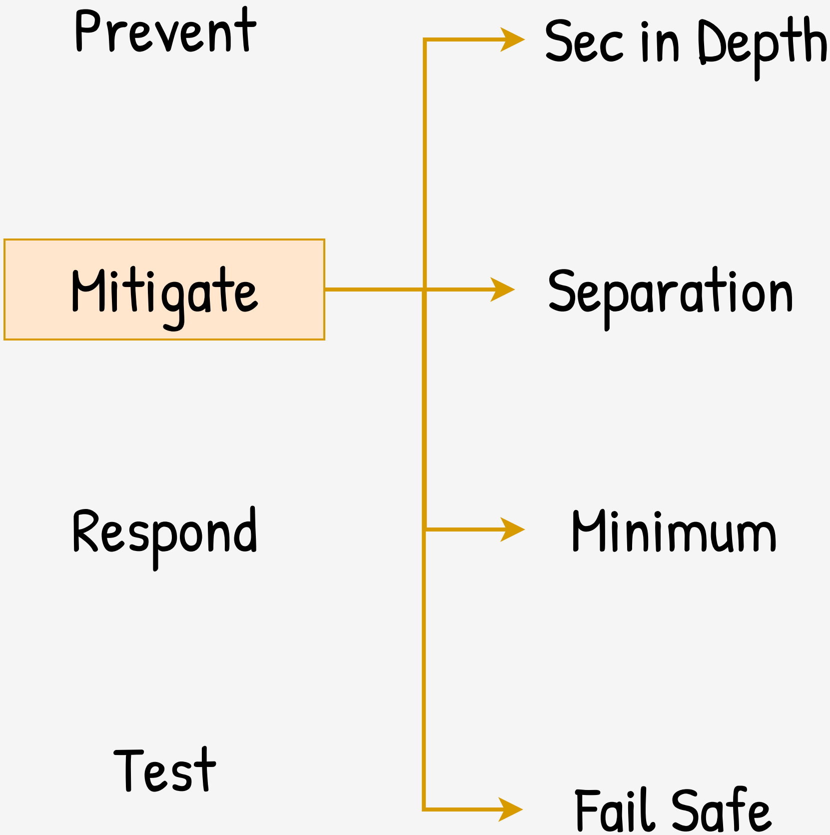
"Pilot" pipe

Main pipe

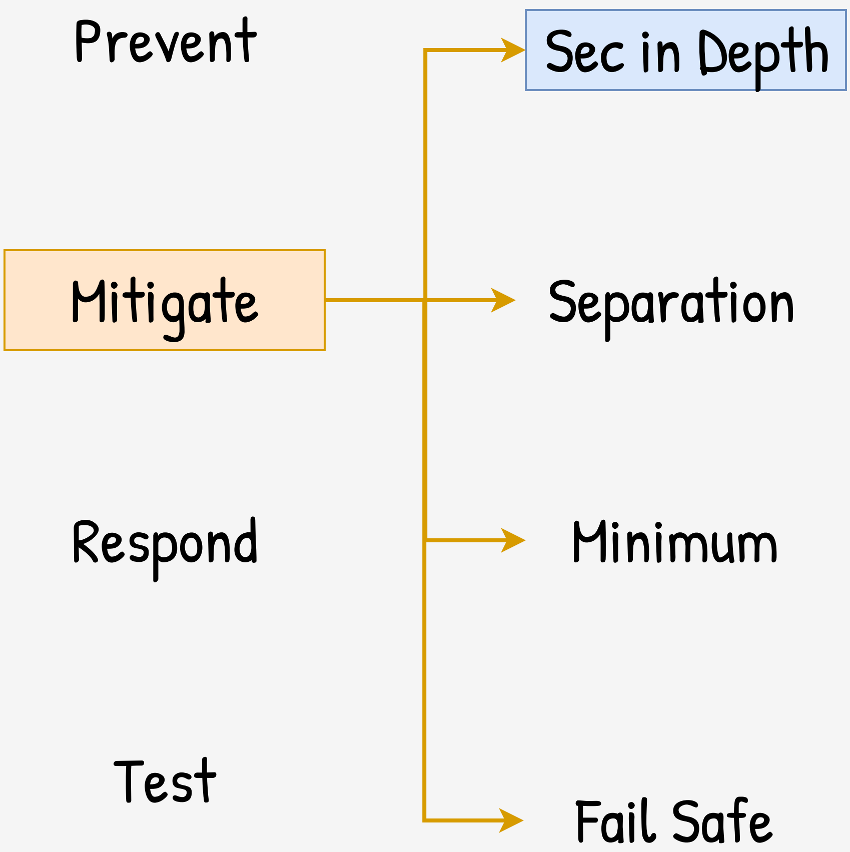


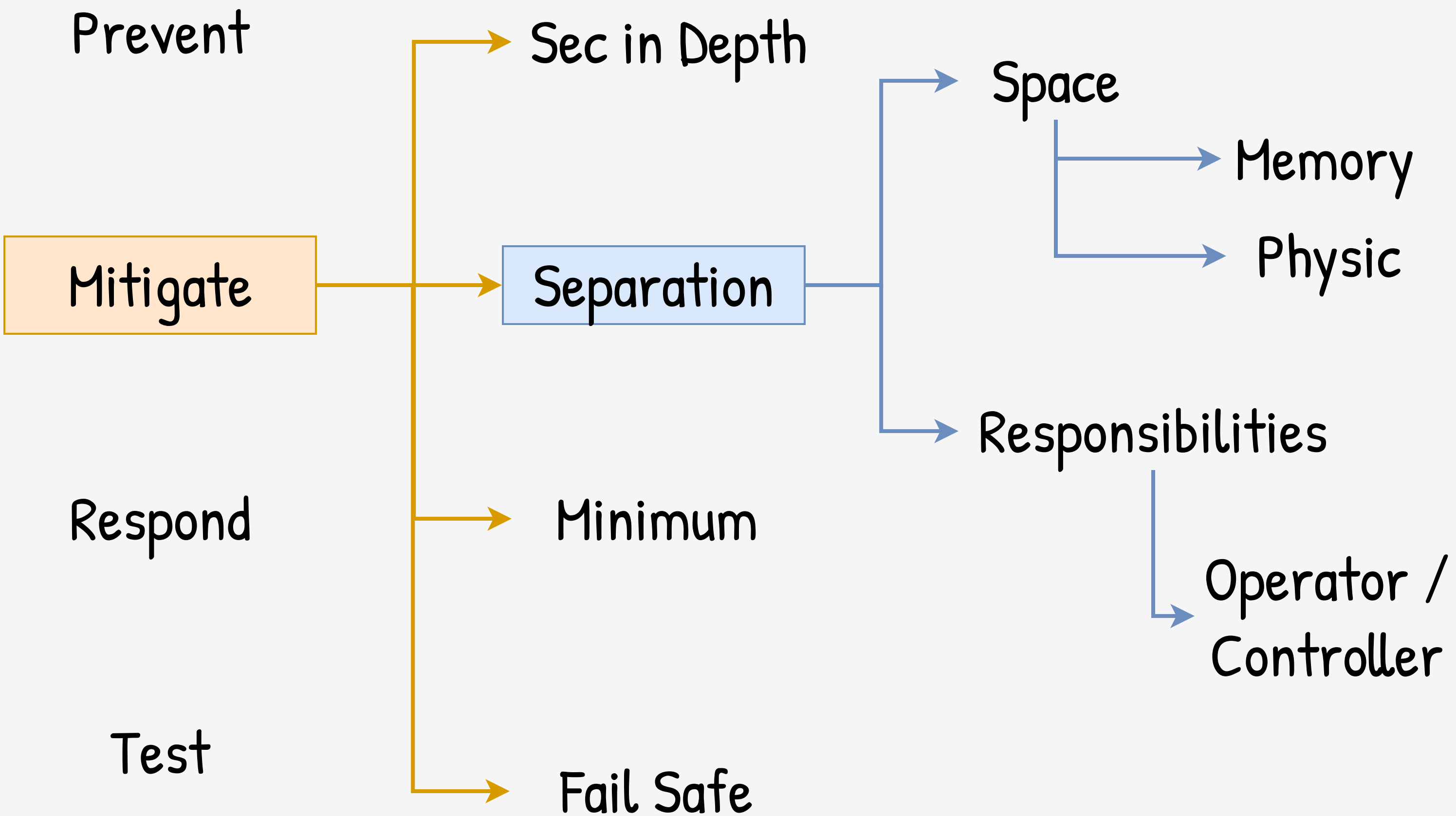
Simple

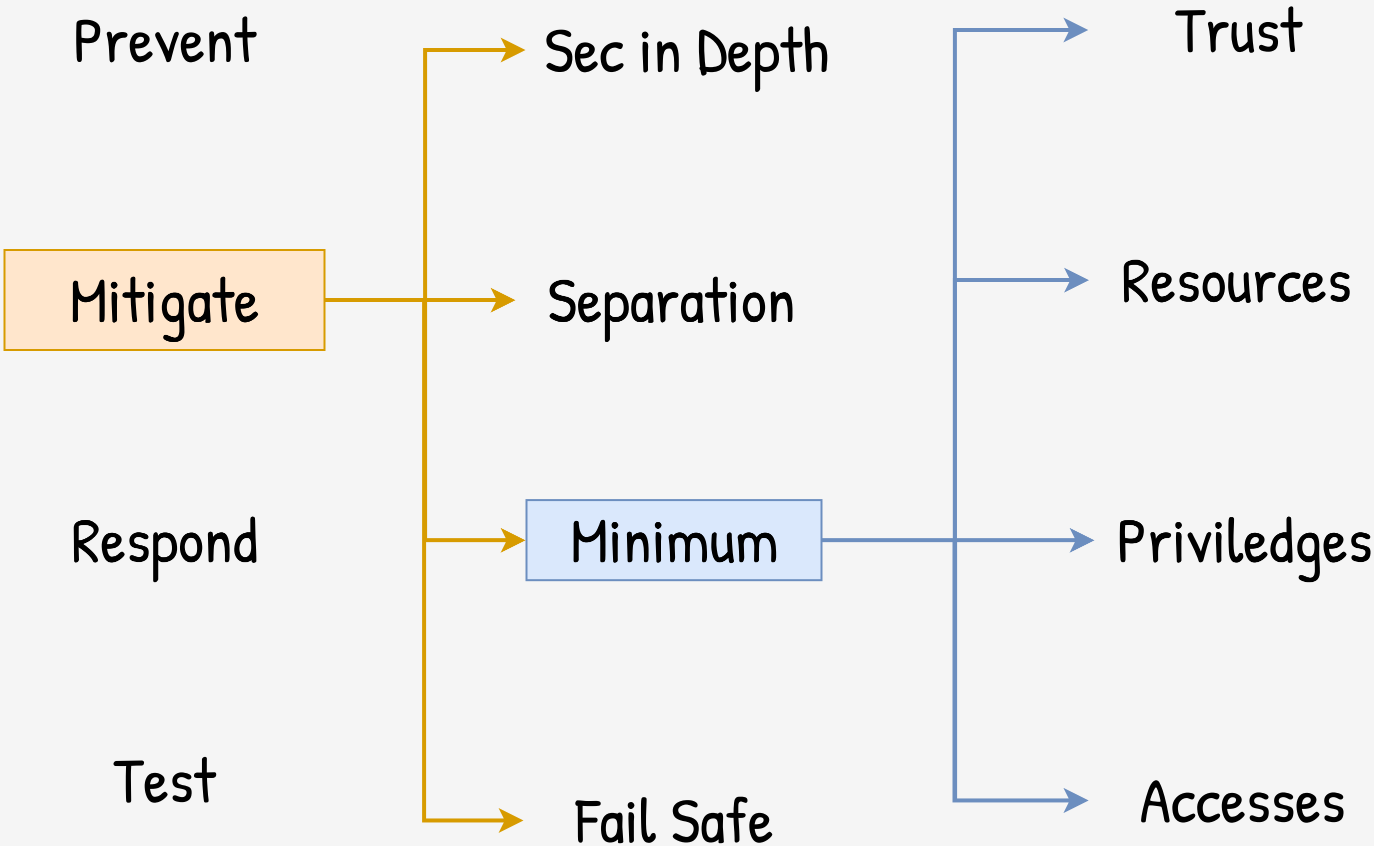
Fail Safe

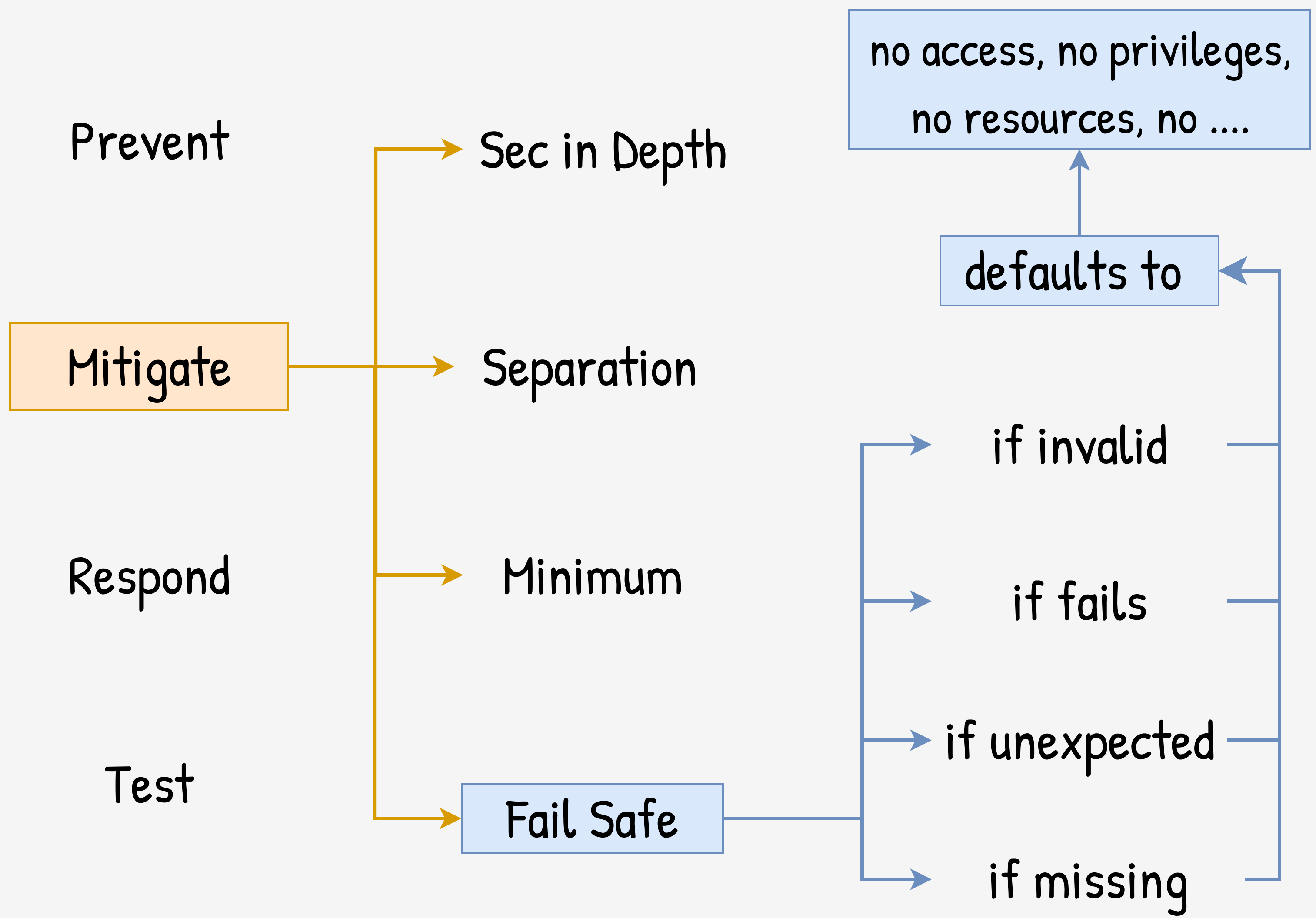


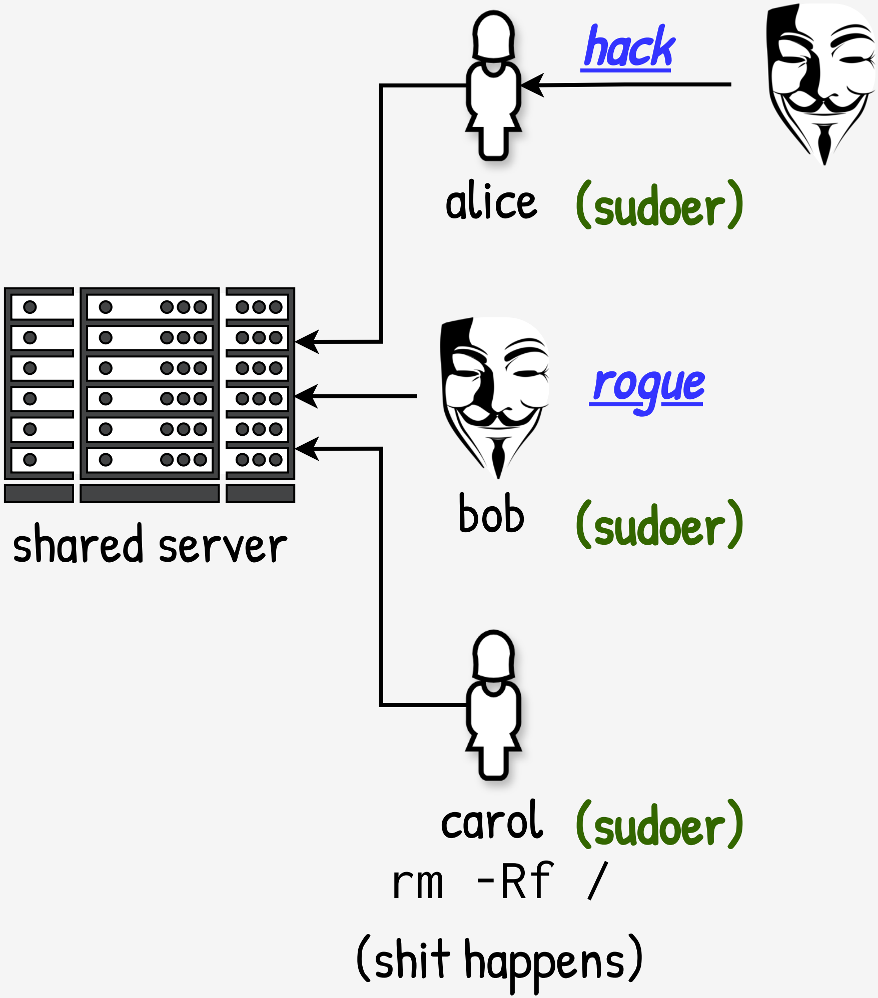
Prevent
Mitigate
Respond
Test
Sec in Depth
Separation
Minimum
Fail Safe


Prevent
Mitigate
Respond
Test
Sec in Depth
Separation
Minimum
Fail Safe

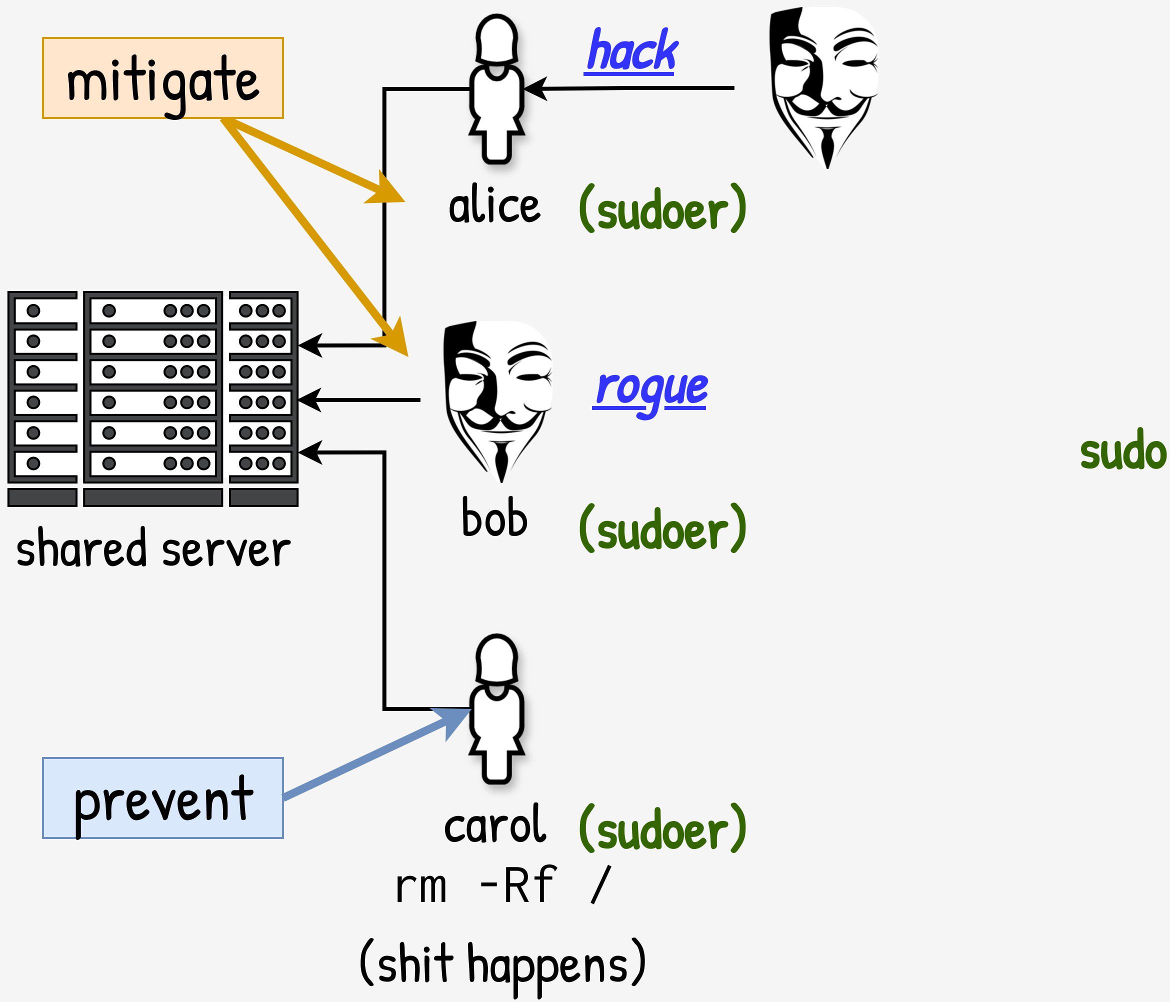

Prevent
Mitigate
Respond
Test
Sec in Depth
Separation
Minimum
Fail Safe
Space
Memory
Physic
Responsibilities
Operator / Controller


Prevent
Mitigate
Respond
Test
Sec in Depth
Separation
Minimum
Fail Safe
Trust
Resources
Priviledges
Accesses

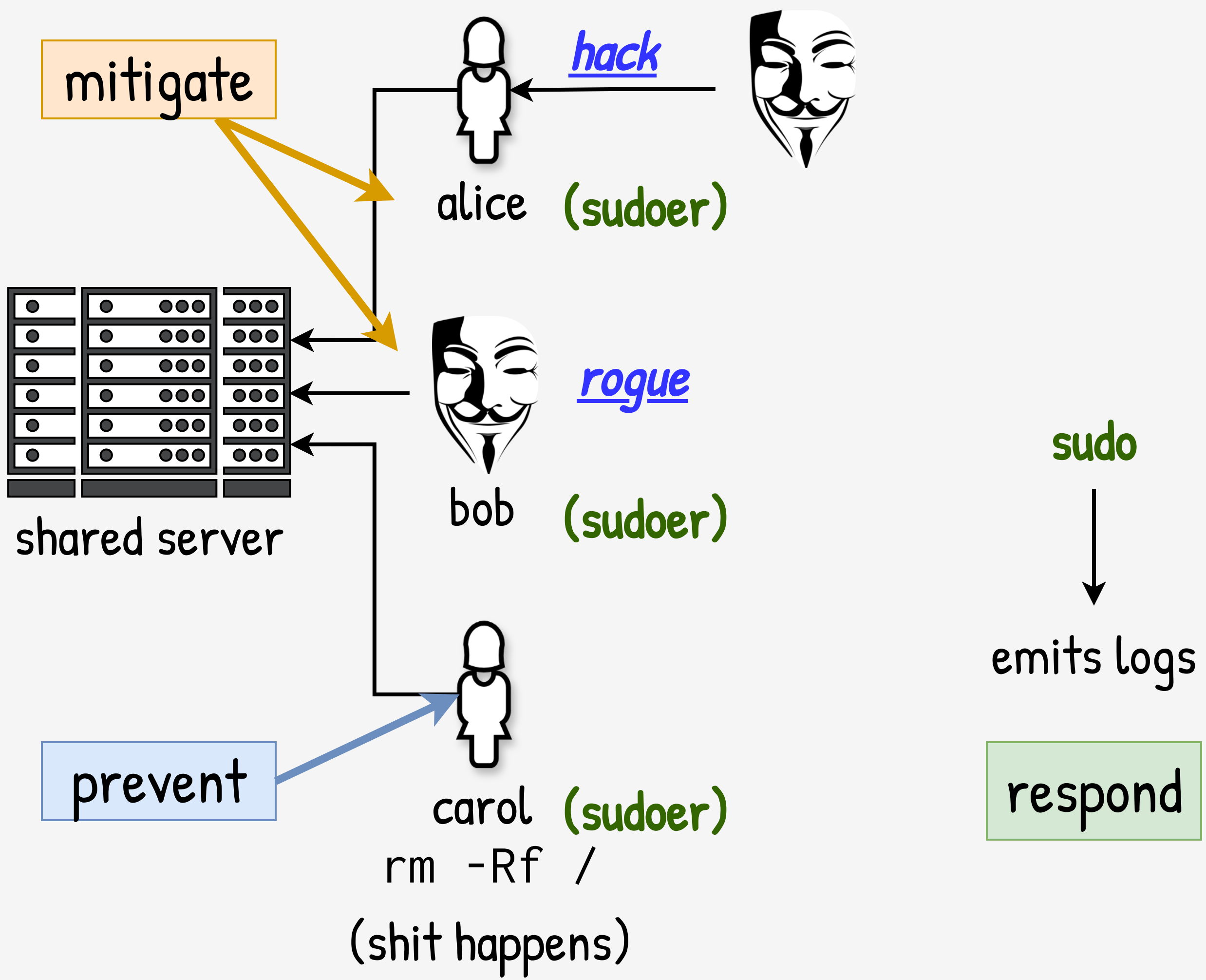

no access, no privileges,
no resources, no ....
Prevent
Sec in Depth
defaults to
Mitigate
Separation
if invalid
Respond
Minimum
if fails
if unexpected
Test
Fail Safe
if missing

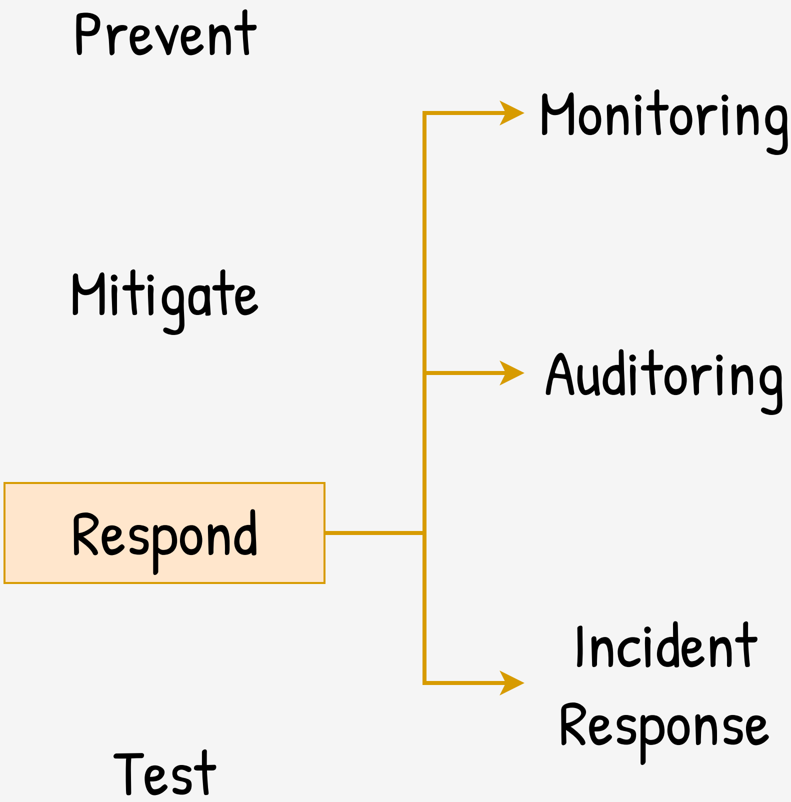

hack
alice
(sudoer)
rogue
bob
(sudoer)
shared server
carol
(sudoer)
rm -Rf /
(shit happens)

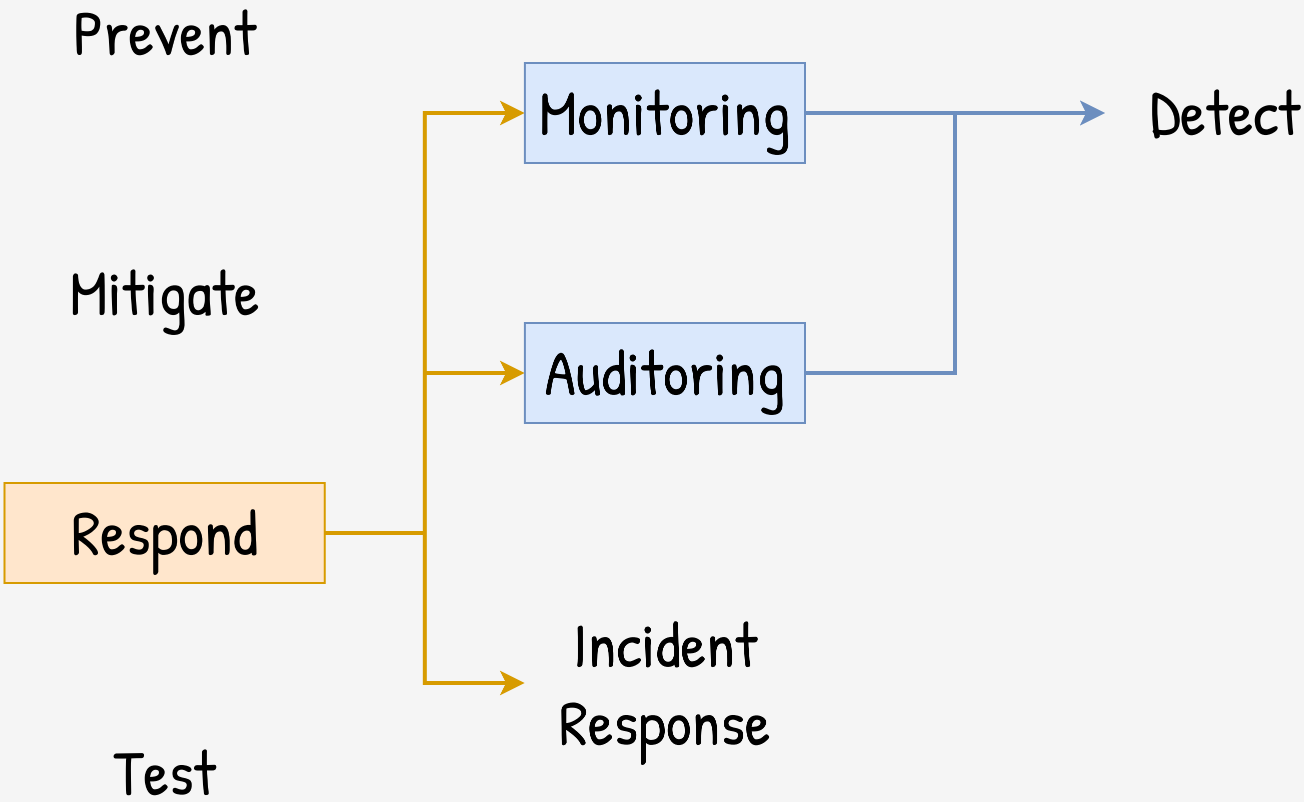
sudo

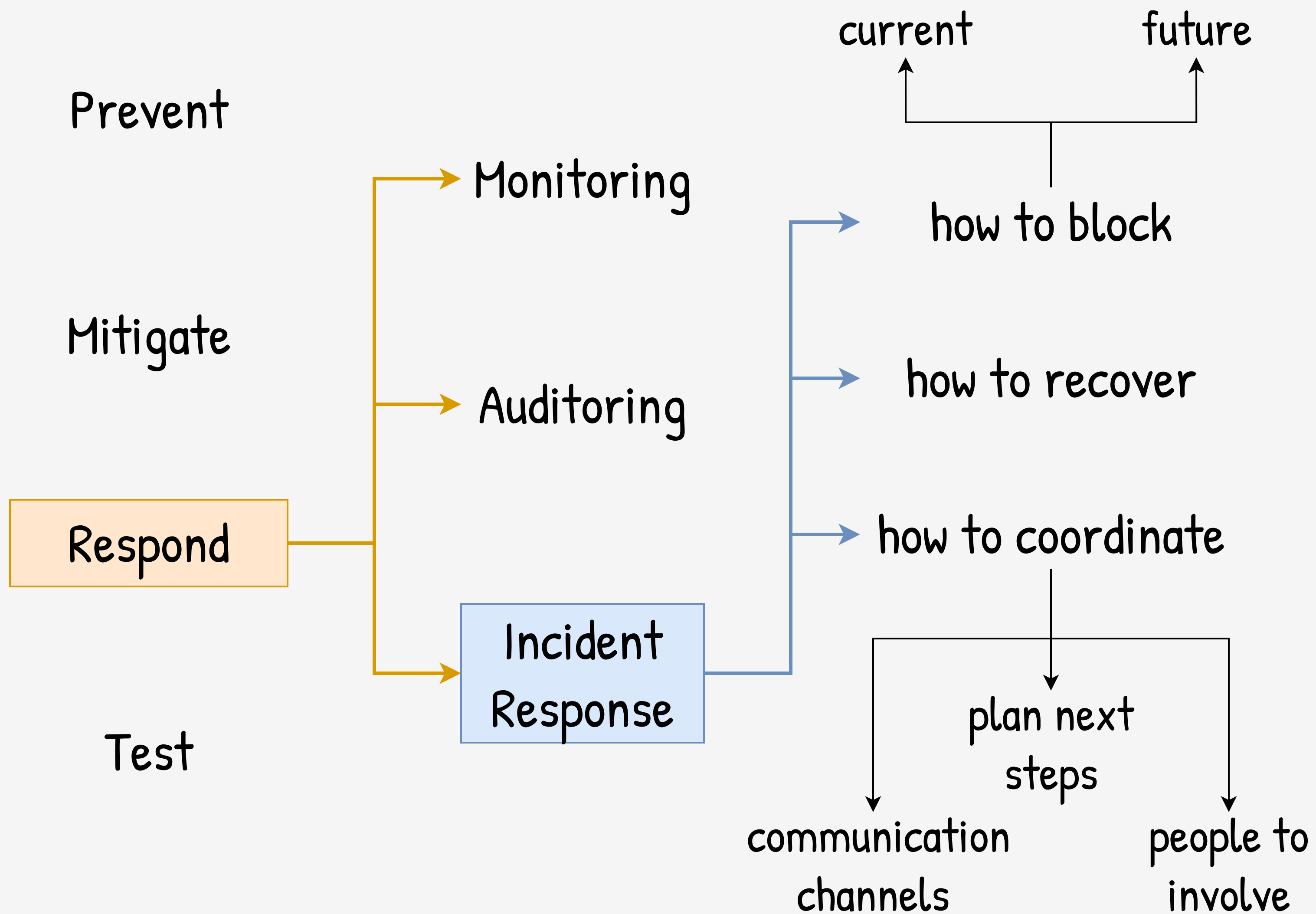


mitigate
hack
alice
(sudoer)
rogue
bob
(sudoer)
shared server
sudo
prevent
carol
(sudoer)
rm -Rf /
(shit happens)

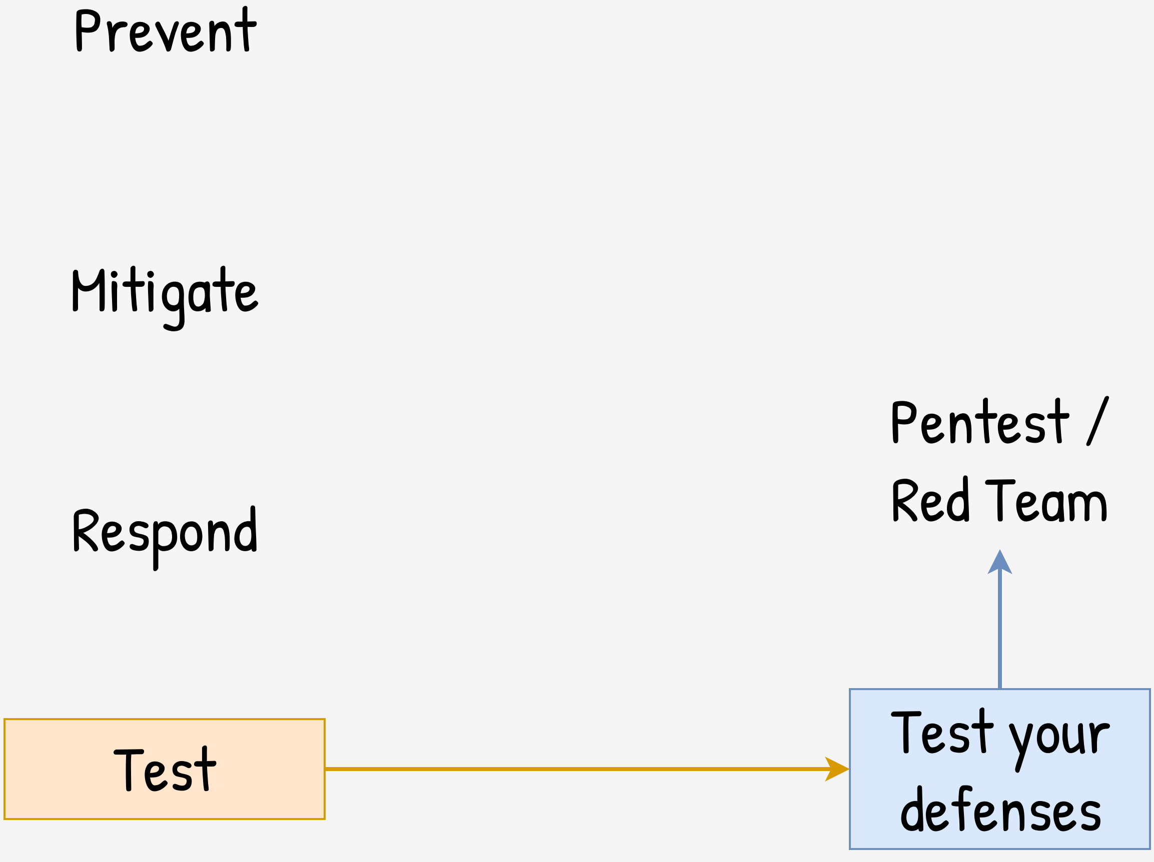

mitigate
alice
hack
(sudoer)
shared server
bob
rogue
(sudoer)
sudo
emits logs
carol
(sudoer)
rm -Rf /
(shit happens)
prevent
respond

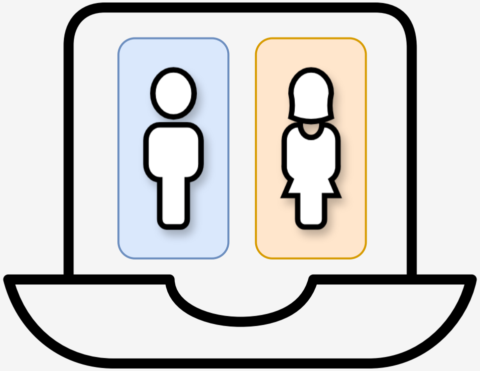

Prevent
Mitigate
Respond
Test
Monitoring
Auditing
Incident
Response


Prevent
Monitoring
Detect
Mitigate
Auditing
Respond
Incident
Response
Test



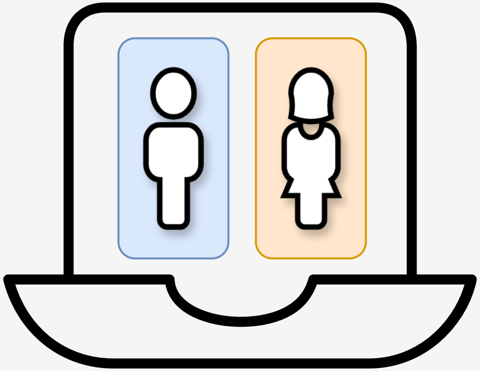
current
future
Prevent
Monitoring
how to block
Mitigate
how to recover
Auditoring
Respond
how to coordinate
Incident Response
plan next steps
Test
communication channels
people to involve


Prevent
Mitigate
Respond
Test
Pentest / Red Team
Test your defenses



voting machine

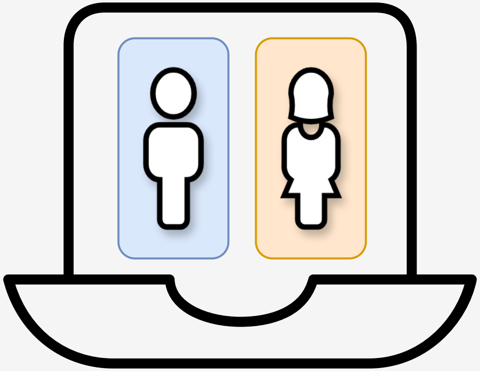
Think Like an Attacker



voting machine

Think Like an Attacker

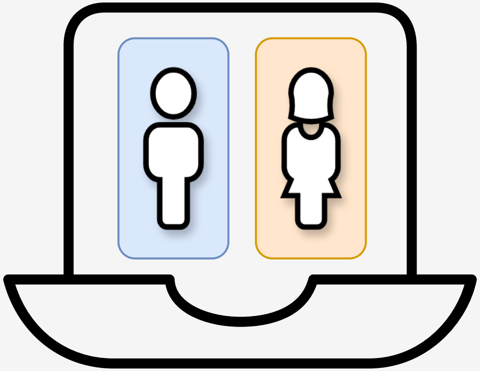
How does it work?



voting machine

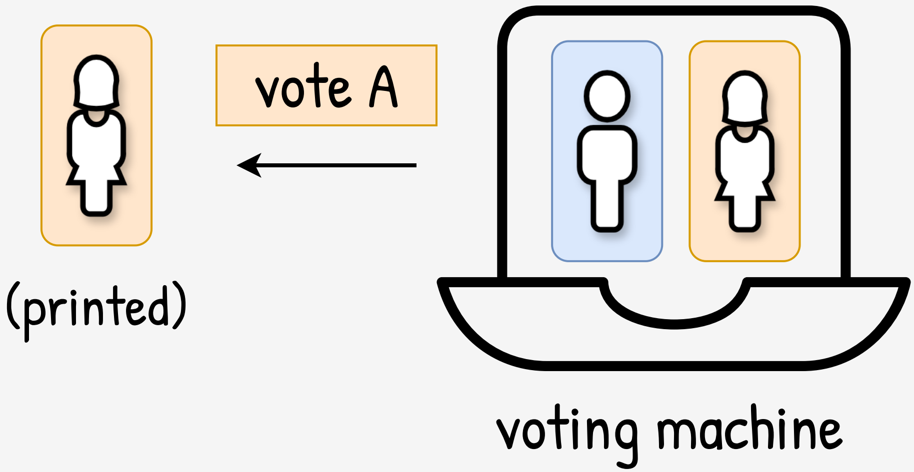
Think Like an Attacker

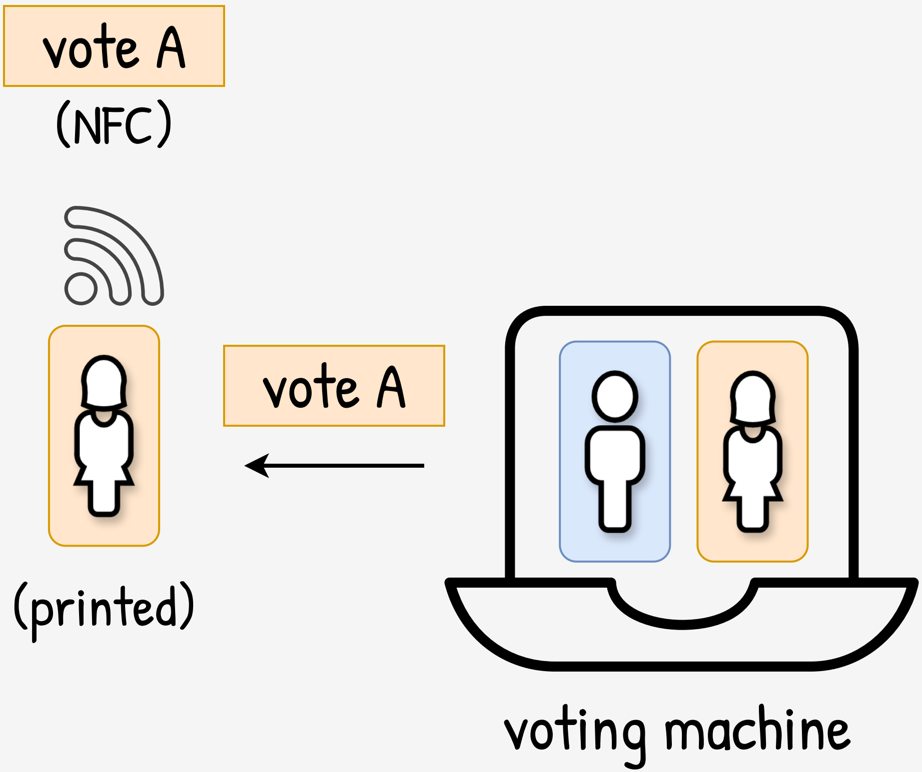
How does it work?

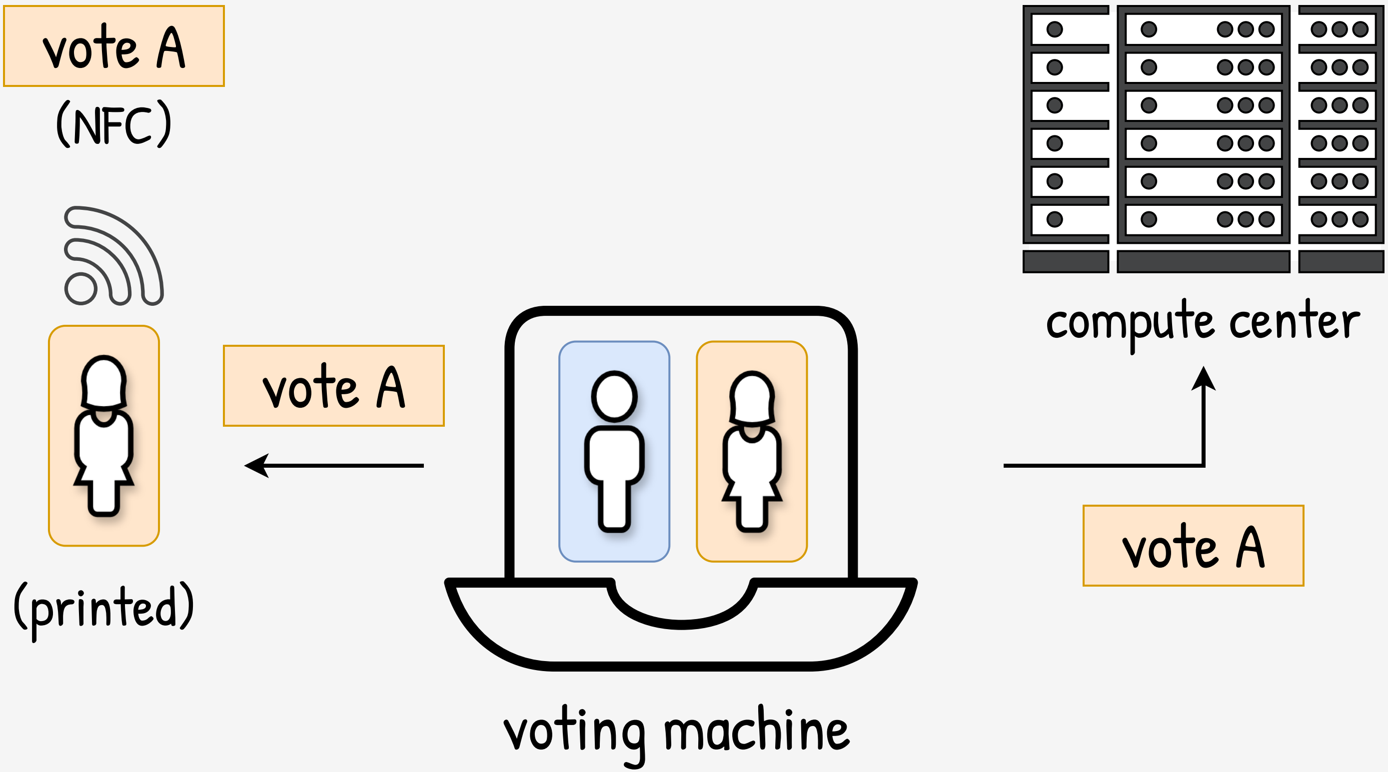


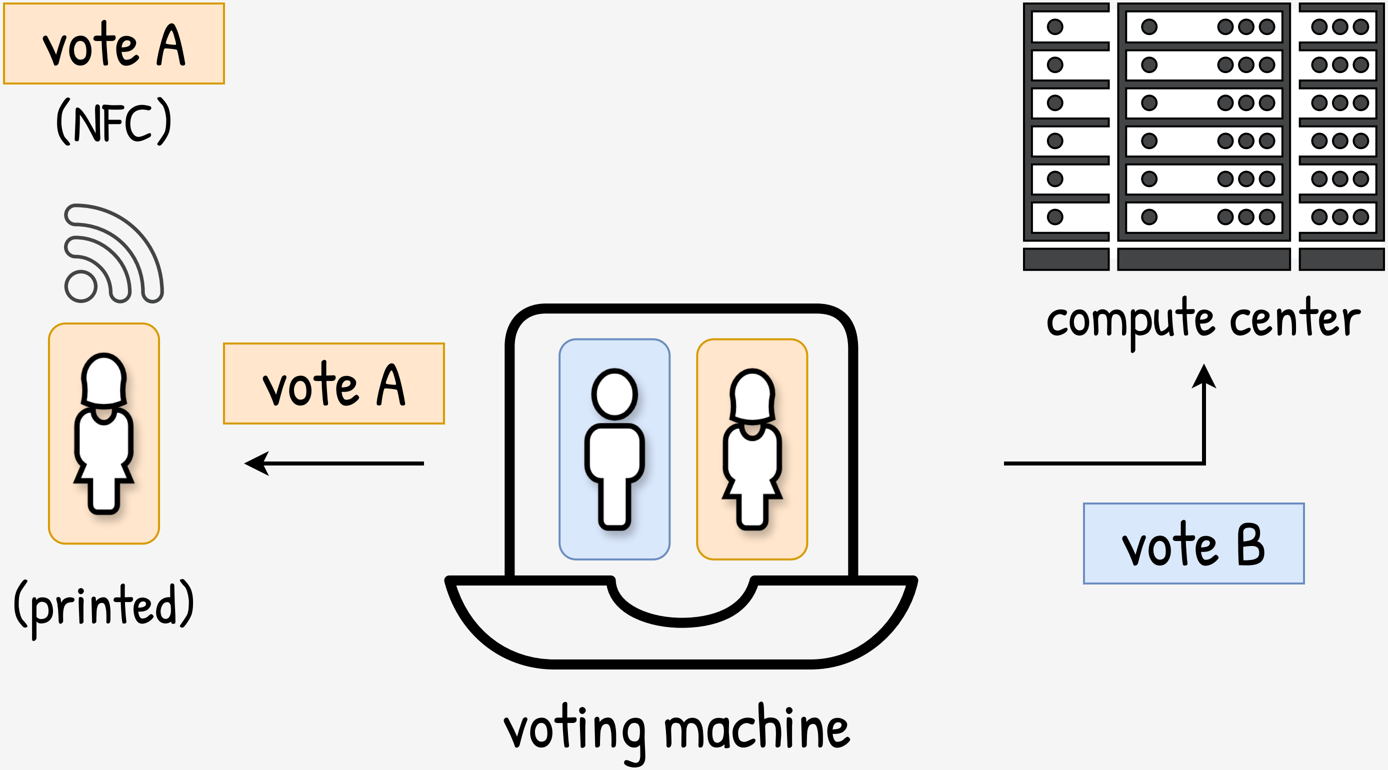
voting machine

How can we hack it?

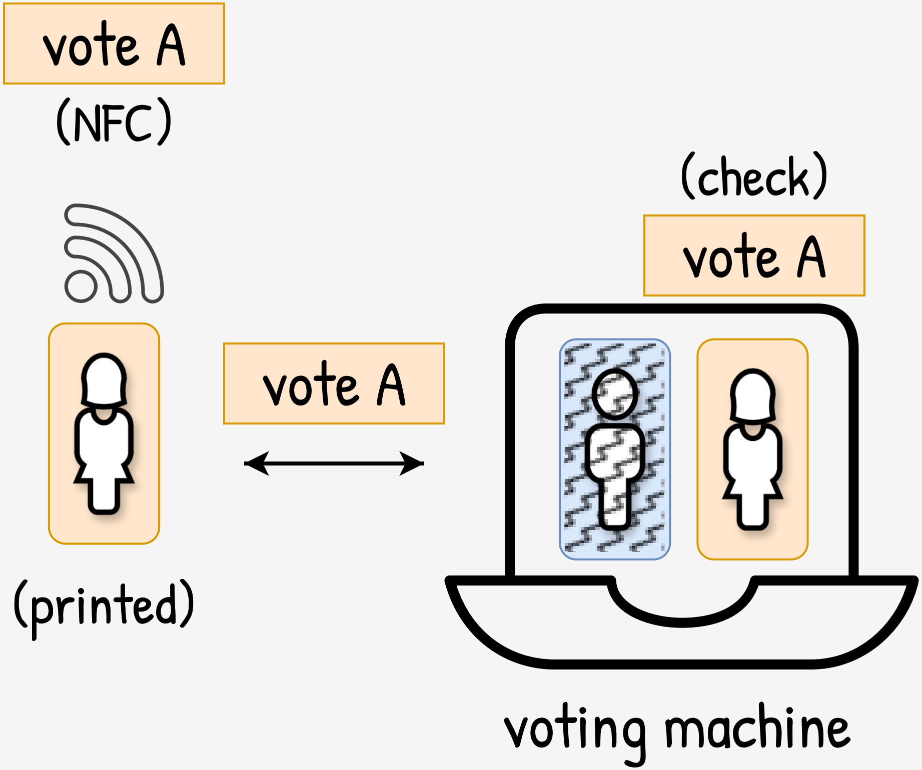

vote A
(printed)
voting machine

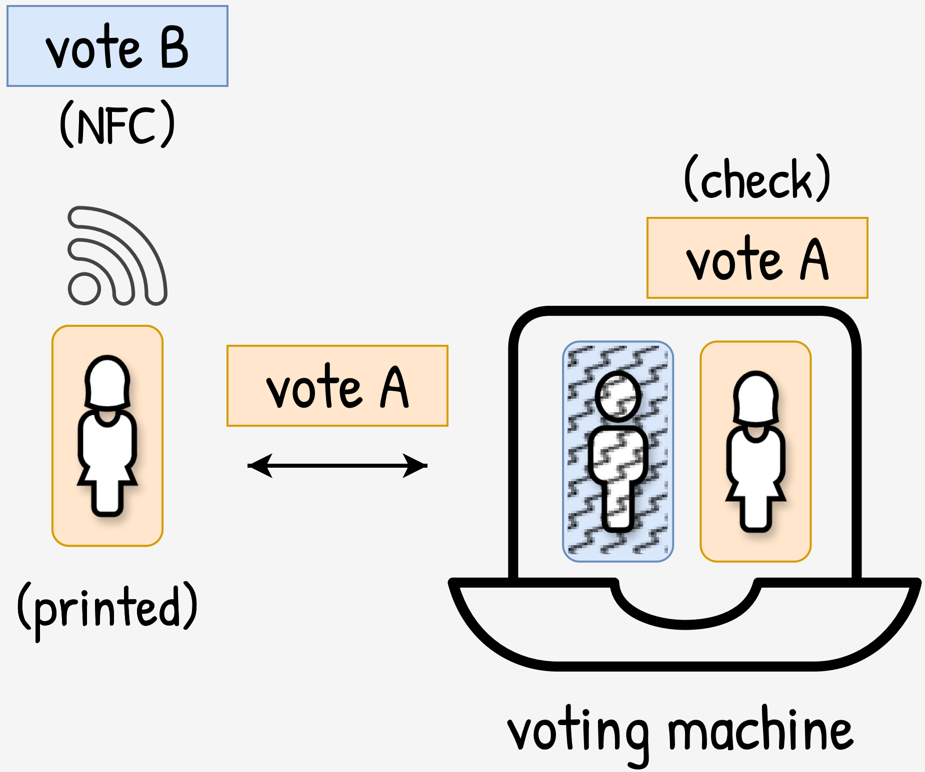

vote A
(NFC)
vote A
(printed)
voting machine


vote A
(NFC)
(printed)
vote A
voting machine
compute center
vote A



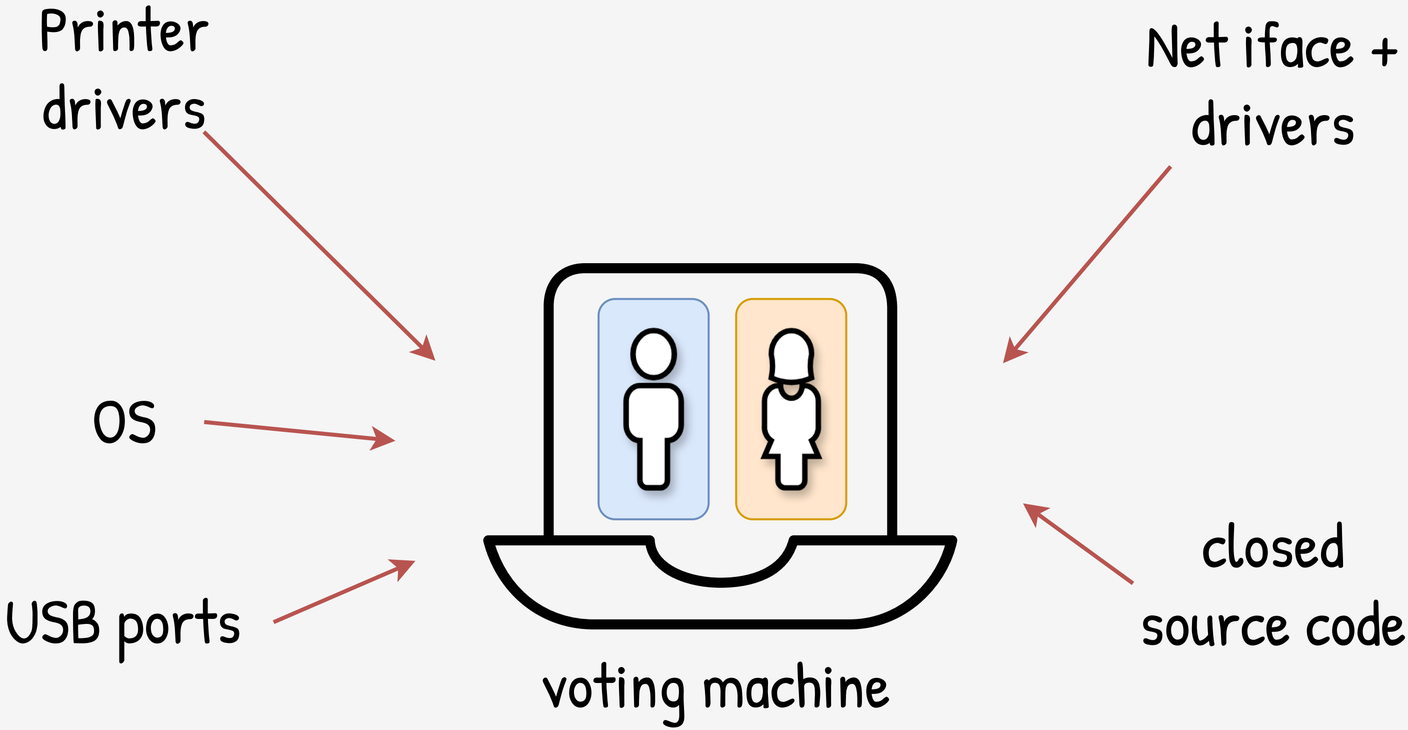
__Opaque__ (not open).
How the voter could *__check__*?
How could she *__trust__*?


vote A
(NFC)
(check)
vote A
vote A
(printed)
voting machine

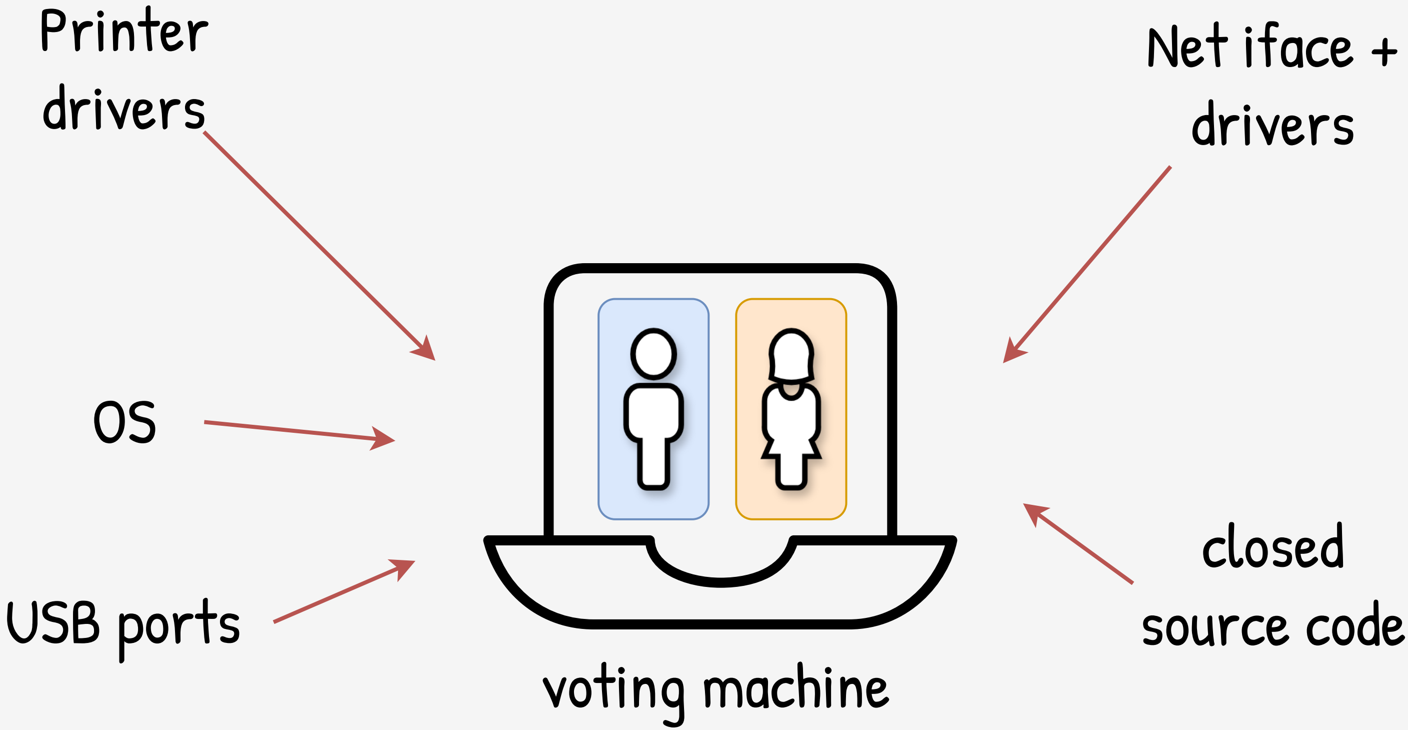


<u>No role separation:</u>

<u>Same</u> machine for voting ***<u>and</u>*** for controlling the vote



https://www.youtube.com/watch?v=HwyN6P83HcY



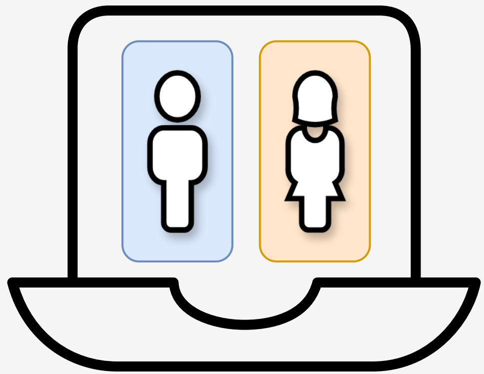
https://www.youtube.com/watch?v=HwyN6P83HcY

Complex (not simple).
How anyone can ***verify*** that the machine ***work as it should***?

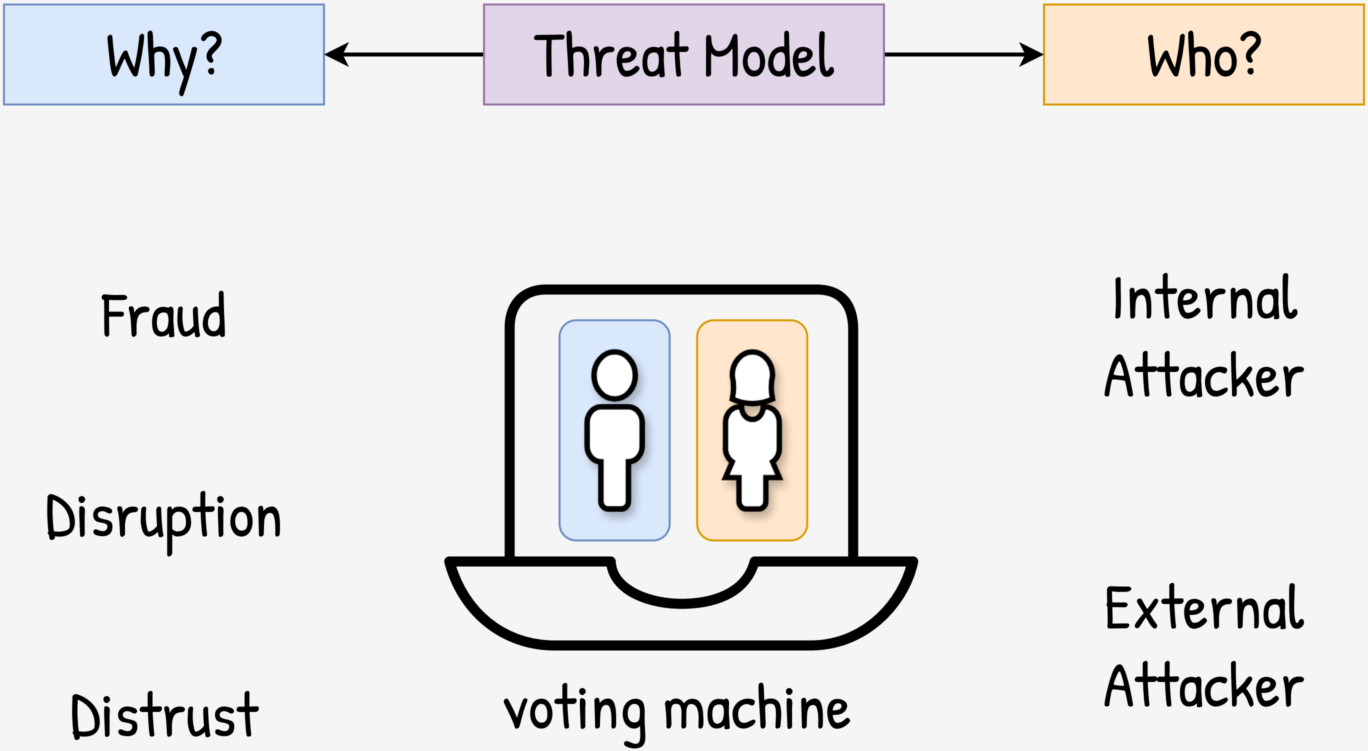
Why? ← Threat Model → Who?

Fraud

Internal Attacker



voting machine

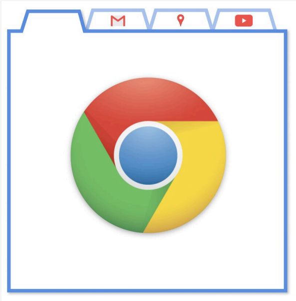

Why?
Threat Model
Who?
Fraud
Disruption
Distrust
voting machine
Internal Attacker
External Attacker

Takeaways

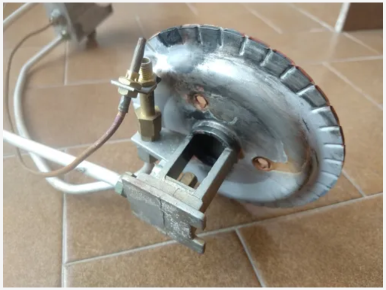
Security goes ***beyond*** software

Defense is not just against an external hack, it's against ***real shit things*** happen

Software



Hardware



People